quadsim: A Modular, Validated Drone Reinforcement-Learning Research Framework on NVIDIA Isaac Lab

Technical Report

Drone_Sim_RL project

November 2026

Abstract

We present *quadsim*, a GPU-parallel quadrotor simulation and reinforcement-learning (RL) platform built on Isaac Lab v2.3.2 and Isaac Sim 5.1. The platform couples a framework-agnostic PyTorch model of multirotor physics (asymmetric first-order motors, rotor and body drag, gyroscopic and reaction torques, ground effect, wind) with PhysX rigid-body integration, models the onboard sensing chain (IMU, barometer, state estimate, depth/RGB cameras, range sensing) including noise, bias and latency, and exposes hover, trajectory tracking, waypoint and vision-based obstacle-avoidance tasks with deployable observations only. Every component is validated against analytic solutions or an independent integrator: PhysX trajectories match the reference integrator to 0.4 mm over 1.5 s of open-loop flight, RTX-rendered depth matches our analytic GPU ray caster to 7 mm median error, and the exported ONNX policy, driven by a NumPy deployment interface, reproduces simulator observations and actions to 10^{-6} in closed loop. PPO policies trained in 10–40 minutes on an 8 GB consumer GPU outperform classical cascade and geometric controllers under identical noise, latency and randomization (hover 99.9 vs 88.7 % success, tracking 12.6 vs 46.6 cm RMSE); a 360° range-sensing navigation policy reaches 83 % success in a dense forest, and a depth-camera policy trained on analytic depth transfers without degradation to RTX-rendered depth (61 vs 67 % success). Negative results (no significant success gain from domain randomization on hover, an under-trained SAC baseline) are reported alongside. Finally, the exported policies were flown as PX4 v1.16 offboard controllers in software-in-the-loop simulation (Gazebo x500) through a ROS 2 bridge: hover, tracking and waypoint policies transfer at the control level (hover 0.8 cm to 1.0 cm, tracking 2–3 cm RMS w.r.t. the onboard estimate, as in quadsim), while the cross-check exposed four bridge defects, a heave limit cycle caused by the thrust-map nonlinearity (removed by an exact thrust-curve map), PX4 EKF2's correlated estimate error, an allocation-geometry asymmetry of the rate loop, and a velocity-loop mismatch that limits the velocity-interface navigation policy (2/6 vs 87 % success; 4/6 after matching PX4's velocity loop). A first SITL-informed training distribution made the heave mode worse (negative result); a corrected one (3 seeds) removed the limit cycle in all 9 SITL flights at the cost of vertical damping and out-of-distribution robustness. Building on this validated core, quadsim was turned into a general research framework (Section 8): registries and plug-ins for vehicles (YAML files, any rotor count), control interfaces (motor-level to position control), algorithms and benchmarks; configuration terms for sensors (rate, latency, history), rewards, terminations, costs, randomization events and curricula; GPU-native SAC/TD3 with asymmetric critics; PPO-Lagrangian constrained RL; parameter-shared multi-agent training with a centralized critic; drone racing and language-conditioned navigation tasks; and an imitation-learning / vision-language-action data path (HDF5 demonstrations, behaviour cloning, DAgger, LeRobot v3.0 export, VLA adapters). A regression check shows the refactored core reproduces 59 reference metrics bit-exactly; the new SAC matches PPO on hover (99.9 %) where the earlier SAC failed; the Lagrangian keeps a speed constraint within its budget; and language-conditioned behaviour cloning reaches the expert's 100 % after two DAgger rounds, but only ∼60 % for unseen phrasings. We report what is verified and, explicitly, what is not (no hardware flight yet), and give a prioritized roadmap.

Contents

1 Introduction and scope **3**

2 Framework selection and related work **3**

3 System architecture **3**

4 Mathematical models **4**
4.1 Frames and rigid-body dynamics . . . 4
4.2 Rotors and motors . . . 4
4.3 Aerodynamics . . . 4
4.4 Wind and disturbances . . . 5
4.5 Control allocation and classical controllers . . . 5
4.6 Sensor models . . . 5

5 Reinforcement-learning environments **5**
5.1 Observation and action design . . . 5
5.2 Tasks and rewards . . . 6
5.3 Domain randomization . . . 6
5.4 Learning algorithm . . . 6

6 Validation and experiments **6**
6.1 M1: simulator foundation . . . 6
6.2 M2: quadrotor dynamics . . . 7
6.3 M3: sensors, recording and visualization . . . 7
6.3.1 Upstream issue found and patched . . . 8
6.4 M4: state-based reinforcement learning . . . 8
6.5 M5: obstacle-avoidance navigation (range and depth) . . . 9
6.6 Domain-randomization ablation . . . 11
6.7 M6: export and deployment pipeline . . . 12
6.8 Off-policy baseline: SAC . . . 12
6.9 M7: throughput, memory and containerization . . . 12

7 PX4 SITL validation and sim-to-sim analysis **12**
7.1 Setup and deployment pipeline . . . 13
7.2 Defects found by SITL . . . 13
7.3 Estimator . . . 14
7.4 Hover and set-point steps . . . 14
7.5 Training distributions v1 / v2 and multi-seed comparison . . . 16
7.6 Trajectory tracking and waypoints . . . 16
7.7 Obstacle navigation through the velocity interface . . . 17
7.8 Summary of the cross-check . . . 18

8 From platform to research framework **19**
8.1 Extension points . . . 19
8.2 Validation of the framework . . . 21
8.3 Findings . . . 21

9 Limitations **24**

10 Roadmap toward real-world deployment **25**

11 Reproducibility **25**

1 Introduction and scope

Learning-based quadrotor control has reached champion-level performance in drone racing [1] and robust low-level control learned in seconds [2]. Such results rest on simulators whose physics and sensing are close enough to reality, and on disciplined interfaces between policy and vehicle. The goal of this project is a *research-grade* platform with these properties, prioritizing verified functionality over feature count.

Hardware note. The original brief targeted an RTX 5070 (12 GB). The actual machine has an **RTX 5060 with 8 GB** (Blackwell, sm_120), Ubuntu 24.04, an i5-14600KF and 32 GB RAM. All budgets (env counts, image sizes) were chosen for 8 GB.

Validation policy. Each milestone has measurable acceptance criteria that are executed by scripts in `scripts/validation`; results are stored as JSON in `docs/results` and summarized in `docs/PROGRESS.md`. RL results distinguish *training execution* from *demonstrated policy performance*, which is always measured by a separate deterministic evaluation with confidence intervals.

2 Framework selection and related work

Isaac Lab offers GPU-parallel PhysX simulation, tiled RTX rendering and RL integrations. At the time of writing the latest releases were v2.3.2 (Isaac Sim 5.1) and v3.0.0 (Isaac Sim 6.2). We selected **v2.3.2 + Isaac Sim 5.1**: it is the latest stable pair, it is available as an official container (`nvcr.io/nvidia/isaac-lab:2.3.2`), and v3.0 introduces breaking changes (quaternion convention $wxyz \to xyzw$, multi-backend refactor). The physics core of quadsim is framework-agnostic to ease a later migration.

Existing quadrotor components. Isaac Lab's `Isaac-Quadcopter-Direct` task applies a body wrench directly from actions without motor dynamics; the `isaaclab_contrib` multirotor asset (from the Aerial Gym authors [4]) adds a first-order thruster model with an allocation matrix. Aerial Gym [4] and OmniDrones [5] provide broader multirotor suites on Isaac Gym / Isaac Sim. We reused ideas (asymmetric motor time constants, allocation matrices, depth-based navigation in cluttered environments) but implemented our own model to obtain (i) rotor drag, gyroscopic, reaction and ground-effect terms, (ii) unit-testability without a simulator, and (iii) an independent reference integrator for validation. The vehicle parameters follow the PX4 SITL Gazebo model of the Holybro X500 so that results can be cross-checked against PX4 SITL later.

Network constraint. NVIDIA's asset CDN (Nucleus/S3) was effectively unreachable from the development network (the stock quadcopter task did not finish loading its USD within 625 s), and NGC refused direct container pulls (HTTP 403). Consequently all assets are *procedural* and the container base image is fetched through the user's proxy by a small registry client with digest verification.

3 System architecture

The code base is layered (Fig. 1):

- `quadsim.core` (pure PyTorch): parameters, motor/aerodynamic model, reference integrators, controllers, sensor and disturbance models, analytic ray casting (PyTorch and NVIDIA Warp kernels).
- `quadsim.isaac`: procedural USD quadrotor (explicit `MassAPI`: mass, inertia and CoM are written from the parameters, not derived from geometry) and version-gated runtime patches for upstream issues.
- `quadsim.tasks`: Isaac Lab `DirectRLEnv` base environment and tasks, PPO configurations and classical baselines that act through the same action interface.

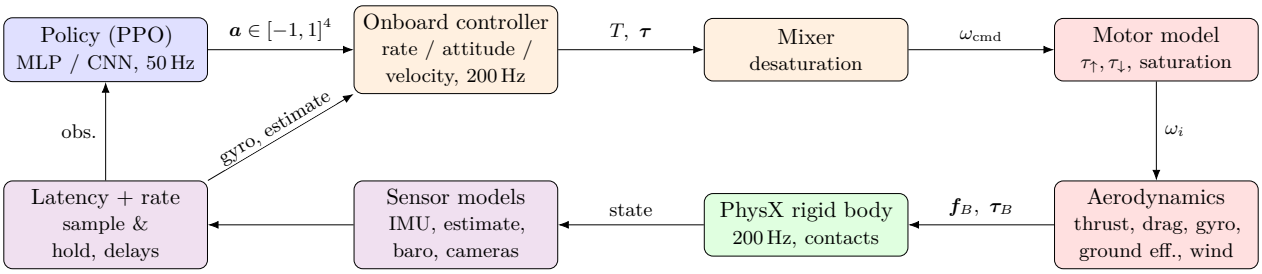


Figure 1: Per-step data flow. Orange, red and violet blocks belong to the framework-agnostic PyTorch core (`quadsim.core`) and are unit-tested without Isaac Sim; the green block is PhysX (Isaac Sim). Rendering (RTX cameras) is an alternative source of image observations.

- `quadsim.rl`, `quadsim.data`, `quadsim.deploy`: networks and policy I/O, HDF5 recording, export and the NumPy deployment interface.

Isaac Lab itself is not modified; it is installed from a pinned tag into `third_party/`.

Simulation timeline. Physics runs at 200 Hz, the policy at 50 Hz (decimation 4). Within each physics step: sensors are sampled; the delayed policy command is passed to the onboard controller (which uses the *noisy* gyro and state estimate); the mixer produces rotor-speed commands; motor states are advanced exactly; the aerodynamic wrench is evaluated and written to PhysX through Isaac Lab's wrench composer (body frame); PhysX integrates the rigid body and contacts.

4 Mathematical models

4.1 Frames and rigid-body dynamics

The world frame W is z-up (ENU-like), the body frame B is FLU, and $\boldsymbol{q} \in S^3$ (scalar first) rotates B into W with rotation matrix $\boldsymbol{R}(\boldsymbol{q})$. With mass m and principal inertia $\boldsymbol{J} = \mathrm{diag}(J_x, J_y, J_z)$,

$$\dot{\boldsymbol{p}} = \boldsymbol{v}, \qquad \dot{\boldsymbol{v}} = \tfrac{1}{m}\boldsymbol{R}\,\boldsymbol{f}_B + \boldsymbol{g} + \tfrac{1}{m}\boldsymbol{f}_W^{\text{ext}}, \tag{1}$$

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\,\boldsymbol{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix}, \qquad \boldsymbol{J}\dot{\boldsymbol{\omega}} = \boldsymbol{\tau}_B - \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}, \tag{2}$$

with $\boldsymbol{g} = (0, 0, -9.81)$. PhysX integrates these equations with a symplectic (semi-implicit) Euler scheme and gyroscopic forces enabled; the reference integrator offers RK4 and the same semi-implicit scheme.

4.2 Rotors and motors

Rotor i at lever arm $\boldsymbol{r}_i$ (relative to the CoM) with spin $s_i = +1$ (CCW) or -1 spins at ω_i:

$$\dot{\omega}_i = \frac{\omega_{\text{cmd},i} - \omega_i}{\tau_i}, \qquad \tau_i = \begin{cases} \tau_\uparrow & \omega_{\text{cmd},i} \geq \omega_i \\ \tau_\downarrow & \text{otherwise} \end{cases}, \qquad \omega_{\text{cmd},i} \in [\omega_{\min}, \omega_{\max}], \tag{3}$$

discretized exactly under zero-order hold: $\omega_i^+ = \omega_{\text{cmd}} + (\omega_i - \omega_{\text{cmd}})e^{-\Delta t/\tau_i}$. Thrust and reaction torque are $T_i = k_f\,\omega_i^2\,g_e(h_i)$ and $Q_i = -s_i\,(k_m T_i + J_r\dot{\omega}_i)$, the latter including the spin-up reaction of the rotor inertia J_r.

4.3 Aerodynamics

With air-relative velocity $\boldsymbol{v}_{\text{rel}} = \boldsymbol{v}_B - \boldsymbol{w}_B$ (wind $\boldsymbol{w}$) and its component $\boldsymbol{v}_{\perp,i}$ orthogonal to the rotor axis at the hub ($\boldsymbol{v}_{\text{rel}} + \boldsymbol{\omega} \times \boldsymbol{r}_i$),

$$\boldsymbol{f}_{\text{rd},i} = -c_{\text{rd}}\,\omega_i\,\boldsymbol{v}_{\perp,i} \quad \text{(blade flapping / induced drag)}, \qquad \boldsymbol{m}_{\text{rm},i} = -c_{\text{rm}}\,s_i\,\omega_i\,\boldsymbol{v}_{\perp,i}, \tag{4}$$

$$\boldsymbol{f}_{\text{body}} = -\boldsymbol{D}_1\boldsymbol{v}_{\text{rel}} - \boldsymbol{D}_2\,|\boldsymbol{v}_{\text{rel}}| \odot \boldsymbol{v}_{\text{rel}}, \qquad \boldsymbol{\tau}_{\text{gyr}} = -\boldsymbol{\omega} \times \textstyle\sum_i J_r s_i \omega_i\,\boldsymbol{e}_z, \tag{5}$$

and the in-ground-effect gain of Cheeseman and Bennett, $g_e(h) = \left(1 - (R/4h)^2\right)^{-1}$, capped at $h \geq R/2$. The total body wrench is

$$\boldsymbol{f}_B = \sum_i \left(T_i \boldsymbol{e}_z + \boldsymbol{f}_{\mathrm{rd},i}\right) + \boldsymbol{f}_{\mathrm{body}}, \quad \boldsymbol{\tau}_B = \sum_i \left[\boldsymbol{r}_i \times \left(T_i \boldsymbol{e}_z + \boldsymbol{f}_{\mathrm{rd},i}\right) + \boldsymbol{m}_{\mathrm{rm},i} + Q_i \boldsymbol{e}_z\right] + \boldsymbol{\tau}_{\mathrm{gyr}}. \tag{6}$$

The X500 preset uses the PX4 Gazebo values ($k_f = 8.549{\times}10^{-6}$, $k_m = 0.016$, $\tau_\uparrow = 12.5\,\mathrm{ms}$, $\tau_\downarrow = 25\,\mathrm{ms}$, $\omega_{\max} = 1000\,\mathrm{rad/s}$, $c_{\mathrm{rd}} = 8.06 \times 10^{-5}$); its composite mass and inertia (base + rotors, parallel-axis theorem) are $m = 2.064\,\mathrm{kg}$, $\boldsymbol{J} = \mathrm{diag}(0.0239, 0.0239, 0.0440)\,\mathrm{kg\,m^2}$, hover speed $769.6\,\mathrm{rad/s}$ and thrust-to-weight 1.69. Body drag coefficients are engineering estimates (not identified).

4.4 Wind and disturbances

Wind is $\boldsymbol{w} = \bar{\boldsymbol{w}} + \boldsymbol{w}_{\mathrm{t}} + \boldsymbol{w}_{\mathrm{g}}$: a per-episode mean, Gauss–Markov turbulence $\boldsymbol{w}_{\mathrm{t}}^{+} = a\boldsymbol{w}_{\mathrm{t}} + \sigma\sqrt{1-a^2}\,\boldsymbol{\eta}$ with $a = e^{-\Delta t/\tau_w}$ (exact discretization of a Dryden-like first-order process with stationary standard deviation σ) and discrete $1-\cos$ gusts (MIL-F-8785C shape) with Poisson arrivals. Push disturbances apply random finite-duration forces.

4.5 Control allocation and classical controllers

The allocation $[F_z, \tau_x, \tau_y, \tau_z]^\top = \boldsymbol{A}\boldsymbol{T}$ with rows $(1,\, y_i,\, -x_i,\, -s_i k_m)$ is inverted with PX4-style sequential desaturation: roll/pitch are scaled only if their spread exceeds the actuator range, the collective is shifted to preserve roll/pitch authority, and yaw is added with the largest feasible scale. The onboard rate controller is $\boldsymbol{\tau} = \boldsymbol{J}(\boldsymbol{K}_p \boldsymbol{e}_\omega + \boldsymbol{K}_i \int \boldsymbol{e}_\omega - \boldsymbol{K}_d \dot{\boldsymbol{\omega}}_f) + \boldsymbol{\omega} \times \boldsymbol{J}\boldsymbol{\omega}$ (derivative on the filtered measurement), the attitude controller is a quaternion P law on the rotation vector of $\boldsymbol{q}^{-1} \otimes \boldsymbol{q}_{\mathrm{sp}}$, and the position controller is a P–PI cascade producing a thrust vector with acceleration and tilt limits. The geometric baseline uses Lee et al.'s SE(3) law [6] $\boldsymbol{f} = m(-\boldsymbol{K}_x \boldsymbol{e}_x - \boldsymbol{K}_v \boldsymbol{e}_v + g\boldsymbol{e}_3 + \ddot{\boldsymbol{p}}_d)$.

4.6 Sensor models

IMU: $\tilde{\boldsymbol{\omega}} = (\boldsymbol{I} + \boldsymbol{M}_g)\boldsymbol{\omega} + \boldsymbol{b}_g + \boldsymbol{n}_g$, $\tilde{\boldsymbol{a}} = (\boldsymbol{I} + \boldsymbol{M}_a)\boldsymbol{R}^\top(\dot{\boldsymbol{v}} - \boldsymbol{g}) + \boldsymbol{b}_a + \boldsymbol{n}_a$, where the specific force is obtained by finite differences of the PhysX velocity (exact for the symplectic scheme), white noise has density σ_c (discrete std $\sigma_c/\sqrt{\Delta t_s}$), and biases are a per-episode turn-on value plus a Gauss–Markov drift with random-walk intensity σ_{rw} (stationary std $\sigma_{rw}\sqrt{\tau/2}$). Default values are the PX4/rotors SITL IMU parameters. Measurements are sampled and held at the sensor rate and pass a per-episode random delay. **State estimate:** $\hat{\boldsymbol{p}} = \boldsymbol{p} + \boldsymbol{b}_p + \boldsymbol{x}_p + \boldsymbol{d}_p + \boldsymbol{n}_p$, $\hat{\boldsymbol{v}} = \boldsymbol{v} + \boldsymbol{x}_v + \boldsymbol{n}_v$, attitude with a multiplicative rotation-vector error, all delayed by a per-episode latency; $\boldsymbol{b}_p \sim \mathcal{N}(0, \mathrm{diag}\,\boldsymbol{\sigma}_b^2)$ is a per-flight bias, $\boldsymbol{x}_p, \boldsymbol{x}_v$ are first-order Gauss–Markov processes ($\dot{\boldsymbol{x}} = -\boldsymbol{x}/\tau + \sigma\sqrt{2/\tau}\,\boldsymbol{w}$, exact discretization $\boldsymbol{x}_{k+1} = a\boldsymbol{x}_k + \sigma\sqrt{1-a^2}\,\boldsymbol{\xi}_k$, $a = e^{-\Delta t/\tau}$), $\boldsymbol{d}_p$ a random walk and $\boldsymbol{n}$ white noise. The v0 tasks use only the white and random-walk terms (EKF/VIO/mocap-like); the parameters of the correlated terms of the v1 tasks were identified from PX4 EKF2 in SITL (Section 7.3). **Barometer:** ISA conversion with noise and Gauss–Markov bias. **Depth camera:** distance-to-image-plane with stereo-like noise $\sigma(z) = \sigma_0 + kz^2$, quantization, per-pixel dropouts, minimum/maximum range; no-return pixels read as maximum range. **Analytic ray casting:** exact ray–plane, ray–finite-cylinder (side and cap) and ray–oriented-box intersections, implemented in PyTorch and as an NVIDIA Warp kernel with $O(NR)$ memory.

5 Reinforcement-learning environments

5.1 Observation and action design

The `policy` observation contains only quantities available onboard: estimated body-frame velocity, the first two columns of the estimated rotation matrix (continuous attitude representation), gyro rates, the previous action and task information expressed in the estimated body frame. The `critic` additionally receives the true state, normalized rotor speeds, wind, disturbance force, and

normalized domain-randomization parameters (asymmetric actor-critic [7]). Four action interfaces are available: individual rotor thrusts, collective thrust and body rates (CTBR, the default for agile tasks following [3]), collective thrust and attitude, and velocity commands (default for navigation, matching PX4 offboard velocity control).

5.2 Tasks and rewards

All rewards are integrated over the policy step Δt; crashes (ground contact, excessive tilt, velocity, distance, or obstacle collision) terminate the episode with a penalty.

- **Hover/recovery**: initial states up to $\pm 1.5\,\mathrm{m}$ offset, 57° tilt, $1.5\,\mathrm{m/s}$, $3\,\mathrm{rad/s}$; reward $1 - \tanh(\|e_p\|/0.8) + 0.5\,e^{-(\|e_p\|/0.1)^2} + 0.2\cos^2(e_\psi/2)$ minus velocity, body-rate and action-rate penalties.
- **Tracking**: references $\boldsymbol{p}_d(t) = \boldsymbol{c} + \sum_k \boldsymbol{A}_k \sin(\boldsymbol{\omega}_k t + \boldsymbol{\phi}_k)$ covering circles, figure-eights and random smooth curves (peak speed $\leq 4\,\mathrm{m/s}$); the policy observes the reference at $t + \{0, 0.1, 0.2, 0.4, 0.6, 1.0\}\,\mathrm{s}$.
- **Waypoints**: five random waypoints (separation $\geq 1.5\,\mathrm{m}$); progress reward $\|\boldsymbol{p} - \boldsymbol{w}\|_{t-1} - \|\boldsymbol{p} - \boldsymbol{w}\|_t$, reach bonus, time penalty.
- **Obstacle navigation**: up to 24 kinematic cylinders ($r \in [0.15, 0.45]\,\mathrm{m}$) rejection-sampled with a minimum gap of $0.9\,\mathrm{m}$, goal 11–12 m ahead; progress, goal, heading-alignment (camera looks where the vehicle flies), altitude-band and proximity terms. A curriculum increases the maximum obstacle count from 8 to 24 when the running success rate exceeds 70 %. Observation variants: 32-ray range ring, analytic depth (64×48, HFOV 87°), RTX depth, RGB, RGB-D.

5.3 Domain randomization

Startup (written to PhysX): mass $\times[0.9, 1.1]$, inertia $\times[0.85, 1.15]$ per axis, CoM offset $\pm 1\,\mathrm{cm}$. Per episode (model side): $k_f \times [0.9, 1.1]$ per rotor, $k_m \times [0.85, 1.15]$, motor time constants $\times[0.7, 1.6]$, $\omega_{\max} \times [0.92, 1]$ (battery sag), drag $\times[0.5, 1.5]$, payload $-5 \ldots + 10\,\%$ of weight, sensor-noise scale $\times[0.5, 1.5]$, action delay 0–2 and estimator delay 0–2 physics steps. The *v1* tasks (`Quadsim-{Hover,Track,Waypoint}-v1`) add the corrections found in the PX4 SITL cross-check (Section 7): the EKF-like estimate model, action and estimator delays of 0–4 steps (0 ms to 20 ms), noise scale $\times[0.5, 2]$, a per-episode rate-PID gain scale $\times[0.6, 1.4]$ and a thrust-map gain around hover, $C = C_h + g\,(C_{\mathrm{cmd}} - C_h)$ with $g \in [0.8, 1.7]$, which represents a thrust-curve or calibration error of the deployed thrust mapping. `Quadsim-Hover-v2` additionally restores the v0 white-noise level and penalizes the change of the collective command (Section 7.5).

5.4 Learning algorithm

PPO [8] (rsl_rl 3.1.2) with GAE ($\gamma = 0.99, \lambda = 0.95$), adaptive learning rate (KL target 0.01), 32 steps per environment, empirical observation normalization. State policies: actor MLP 256–256–128, critic 512–256–128 (ELU). Vision policies: a 3-layer CNN encoder (32/64/64 channels) to a 128-d feature, fused with the normalized state in a 256–128 actor; the critic is a state-only MLP on privileged inputs.

6 Validation and experiments

Table 1 summarizes the acceptance status of every milestone; the subsections give the evidence.

6.1 M1: simulator foundation

Sixty-four rigid bodies in free fall for 0.5 s at $\Delta t = 2.5\,\mathrm{ms}$ end at $|z - z_{\mathrm{symplectic}}| = 9.5 \times 10^{-7}\,\mathrm{m}$ (float32 resolution) and $|z - z_{\mathrm{analytic}}| = 6.1\,\mathrm{mm}$, which is exactly the expected first-order bias $g\,t\,\Delta t/2$; this identifies PhysX's integrator as symplectic Euler, which the reference simulator reproduces. The same test passes inside the Docker container.

Milestone	Acceptance criterion (executed)	Status
M1 Isaac Sim/Lab	headless GPU boot; PhysX free fall vs symplectic Euler $< 1\,\text{mm}$ (got $9.5 \times 10^{-7}\,\text{m}$)	**pass**
M2 Dynamics	unit tests (all modules); PhysX vs reference $< 1\,\text{cm}/1.5\,\text{s}$; inertia exact; motor/yaw models $< 3\,\%$	**pass**
M3 Sensors, recording, visualization	sensor statistics; rendered vs analytic depth (median $< 2\,\text{cm}$); HDF5 round trip; GUI and video	**pass**
M4 State-based RL	training completes with checkpoints; evaluated vs classical baselines	**pass**
M5 Vision RL	depth policy trained and evaluated (incl. rendered depth); RGB/RGB-D pipelines execute	**pass***
M6 DR & Sim-to-Real interfaces	DR ablation; ONNX = PyTorch to 10^{-5}; deployment-in-the-loop; ROS 2/PX4 bridge	**pass**†
M7 Docker, benchmark, docs	container runs tests + GPU smoke test; benchmark table; README; report	**pass**
PX4 SITL	policies fly PX4 v1.16 SITL as offboard controllers; quadsim-vs-SITL comparison of hover, steps, tracking, waypoints, navigation; bridge safety gating verified	**pass**‡

Table 1: Milestone acceptance summary. *Criteria met; the depth policy's success (61–67 %) is moderate and no RGB(-D) policy performance is claimed. †DR showed no significant success gain on hover; the bridge was validated in PX4 SITL afterwards. ‡Hover, tracking and waypoint transfer at the control level; the velocity navigation policy needs a matched velocity loop (Section 7).

6.2 M2: quadrotor dynamics

Unit tests (CPU, no simulator) verify quaternion algebra, FLU sign conventions of roll/pitch/yaw torques, the exact motor step response (63.2 % after τ, separate rise/fall constants, discretization invariance), rotor and body drag, ground effect and gyroscopic torque, analytic free fall, constant-torque yaw, conservation of angular momentum and energy for a tumbling body (Dzhanibekov effect), first-order convergence of the semi-implicit integrator to RK4, closed-loop position steps with the full cascade ($< 5\,\text{cm}$), recovery of the Crazyflie from 46° tilt and 3.7 rad/s, SE(3) circle tracking ($< 10\,\text{cm}$ RMSE), and allocation/desaturation properties. Table 2 lists the Isaac Sim integration tests; Fig. 2 shows the open-loop comparison.

Test	X500	Crazyflie	Criterion
PhysX mass / inertia rel. error	0e+00 / 0e+00	0e+00 / 0e+00	$< 10^{-4}$
Hover altitude drift over 2 s [mm]	0.00	0.00	< 5
Thrust step: accel. RMS err / peak	0.019 %	0.026 %	$< 2\,\%$
Yaw accel. vs τ_z/J_z	0.0036 %	0.0077 %	$< 3\,\%$
Open loop 1.5 s, pos. err vs semi-implicit ref. [mm]	0.40	2.07	< 10
Open loop, att. err vs semi-implicit ref. [deg]	0.006	0.066	< 0.5
Open loop, pos. err vs RK4 ref. [mm] (PhysX discretization)	3.3	8.9	info
Closed-loop velocity cascade, PhysX vs ref. [mm]	12.3	9.2	< 30

Table 2: PhysX integration of the quadrotor model vs analytic models and the independent reference integrator (`scripts/validation/m2_dynamics_validation.py`). Sensor noise, latency and randomization disabled.

6.3 M3: sensors, recording and visualization

Statistical unit tests confirm the IMU white-noise standard deviation at 200 Hz ($\sigma_c\sqrt{f}$, within 5 %), turn-on bias, exact latency, zero specific force in free fall and the correct gravity direction when tilted, ISA barometric conversion (100 m → 100 129 Pa), Gauss–Markov stationary statistics, depth-noise statistics, exact ray–primitive intersections and agreement of the Warp ray-casting kernel with the

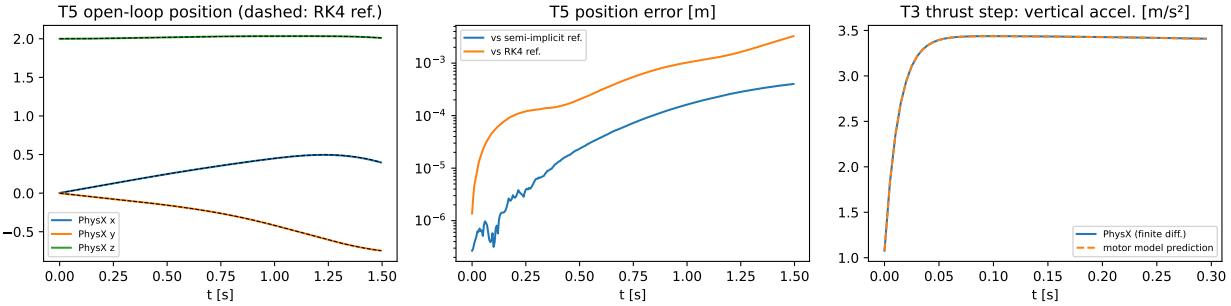


Figure 2: X500: open-loop multisine rotor excitation (left), position error of PhysX vs the reference integrator (middle; semi-implicit reference = same discretization, RK4 = integration error of PhysX at 200 Hz) and thrust-step response (right).

PyTorch implementation. A recording plot revealed a bug (no-return depth pixels received $\sigma \propto z^2$ noise); it was fixed and a regression test added.

Rendered vs analytic depth. For 64 random poses among 24 obstacles (96×72, HFOV 87°, noise off), RTX-rendered and analytic depth agree as in Table 3. Residuals concentrate on silhouette pixels (anti-aliasing) and cylinder tessellation. The comparison also exposed that the initial camera mount saw the X500's front rotor discs (3 % of the image); the default mount was moved to $(0.15, 0, -0.06)$ m, which has no self-occlusion at zero pitch.

Camera pitch	median [mm]	p90 [cm]	hit/miss agree	self-occl.	pixels
0°	7.2	2.4	99.6 %	0.0 %	337361
15°	9.0	2.3	99.5 %	5.6 %	254392

Table 3: RTX TiledCamera depth vs analytic ray-cast depth (`scripts/validation/m3_depth_consistency.py`). Self-occluded pixels (vehicle rotors, rendered < 0.4 m, analytic > 0.45 m) are excluded from the error metrics and reported separately.

Recording. The synchronized HDF5 recorder was verified by round trips for a state task (tracking, SE(3) baseline) and a rendered-depth task (navigation, potential-field baseline): dataset shapes, monotonic timestamps with constant Δt, exact equality of the last sample with the live simulator, and the embedded configuration. **Visualization.** Headless MP4 recording and the interactive GUI viewer (chase camera, target and reference markers) work after patching an upstream Isaac Lab issue (Section 6.3.1).

6.3.1 Upstream issue found and patched

With any rendered viewport (GUI, `-video`), every RigidObject failed to initialize with "Failed to get rigid body transforms from backend" — reproduced with a plain Isaac Lab cuboid. Cause: `WrenchComposer.__init__` uses `hasattr(asset.data, "body_link_pos_w")`; `hasattr` evaluates the property, which queries PhysX before its GPU buffers exist during the PLAY callback, and the raised `Exception` is not an `AttributeError`. quadsim applies a version-gated patch that checks the attribute on the class (no PhysX query), leaving Isaac Lab's sources untouched. Isaac Lab's viewport chase camera has a similar start-up read, which the viewer avoids by enabling asset following only after the first reset.

6.4 M4: state-based reinforcement learning

All policies use CTBR actions at 50 Hz, the full randomization of Section 5.3, sensor noise and latency. Evaluation uses the deterministic policy and identical seeds (hence identical initial states and randomization) for all controllers; ± denotes a 95 % confidence interval of the mean.

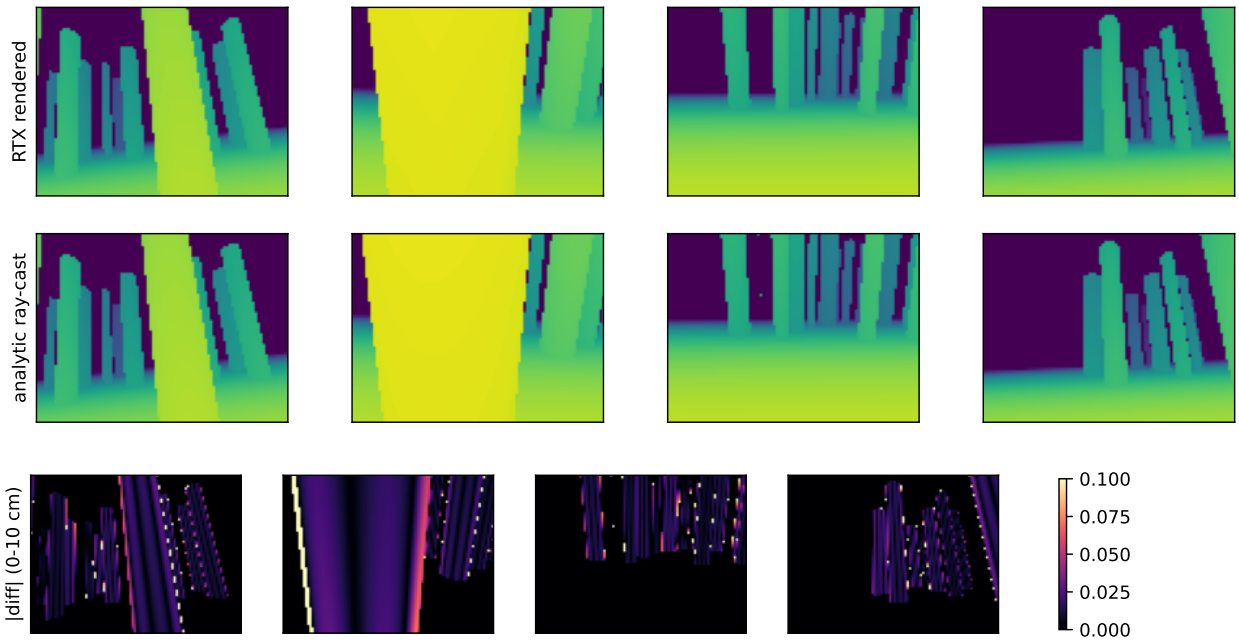


Figure 3: Rendered depth (top), analytic ray-cast depth (middle) and absolute difference clipped at 10 cm (bottom).

Hover and recovery. Training: 4096 environments, 600 PPO iterations (79 M transitions, ≈10 min on the RTX 5060). Success requires $\|e_p\| < 10$ cm and $\|v\| < 0.2$ m/s at the end of the 8 s episode. The policy also generalizes to wind (0–3 m/s mean, turbulence, gusts) and push disturbances that were *not* present during training (Table 4). The policy's yaw accuracy (4.4°) is worse than the baseline's because of the weak yaw reward weight.

Condition	Controller	success [%]	crash [%]	final pos. err [cm]	final yaw err [deg]	episodes
nominal	PPO (ours)	99.9 ± 0.2	0.1 ± 0.2	1.38 ± 0.26	4.4 ± 0.2	1000
nominal	Cascade PID	88.7 ± 2.0	0.5 ± 0.4	6.10 ± 0.74	1.5 ± 0.3	1000
wind + pushes	PPO (ours)	97.3 ± 1.0	0.1 ± 0.2	2.86 ± 0.33	4.6 ± 0.2	1000
wind + pushes	Cascade PID	78.6 ± 2.5	0.7 ± 0.5	8.27 ± 0.94	1.8 ± 0.4	1001

Table 4: Hover/recovery from up to 57° tilt, 3 rad/s, 1.5 m/s and 1.5 m offset (1000 episodes per row).

Trajectory tracking. Training: 4096 environments, 1500 iterations (197 M transitions). The aggressive reference set contains accelerations up to ≈15 m/s^2, partly beyond the X500's sustainable 13 m/s^2 (thrust-to-weight 1.69) and beyond the baselines' tilt limits; the moderate set (≤2 m/s, ≤0.35 Hz) is feasible for all controllers. The first SE(3) baseline, which mapped Lee's torque-level attitude law onto rate set-points at the 50 Hz policy rate, was unstable (53 % crashes, row †); the redesigned baseline uses Lee's position law with the onboard attitude/rate loops.

Waypoint navigation. Training: 4096 environments, 1500 iterations. Five random waypoints (separation ≥ 1.5 m, 0.3 m acceptance radius) must be passed in order within 15 s. The cascade baseline flies to each waypoint with its velocity limits (5 m/s horizontal), facing the waypoint.

6.5 M5: obstacle-avoidance navigation (range and depth)

The navigation task is the hardest: up to 24 cylinders in an 8 × 8 m field with 0.9 m minimum surface gap for a vehicle of 0.35 m collision radius, velocity-command actions and a forward-facing camera (87° HFOV) for the depth policy. Evaluation uses the full density (curriculum off, 4–24 active obstacles, mean ≈12).

Reference set	Controller	RMSE [cm]	max err [cm]	crash [%]	episodes
aggressive (≤4 m/s)	PPO (ours)	12.6 ± 1.5	40.6 ± 3.0	0.2 ± 0.4	500
aggressive (≤4 m/s)	Cascade PID	46.6 ± 3.7	102.8 ± 7.2	3.2 ± 1.5	500
aggressive (≤4 m/s)	SE(3) geometric	49.2 ± 3.9	97.7 ± 6.9	1.8 ± 1.2	500
aggressive (≤4 m/s)	SE(3) v1 (torque-level @50 Hz)†	114.2 ± 5.0	221.8 ± 9.0	53.2 ± 4.4	500
moderate (≤2 m/s)	PPO (ours)	5.8 ± 0.2	29.0 ± 0.8	0.0 ± 0.0	500
moderate (≤2 m/s)	Cascade PID	17.4 ± 1.3	50.2 ± 3.7	0.0 ± 0.0	500
moderate (≤2 m/s)	SE(3) geometric	22.7 ± 1.4	52.8 ± 3.8	0.2 ± 0.4	500

Table 5: Trajectory tracking (500 episodes of 10 s per row). Maximum errors include the initial transient (start offset up to 0.3 m).

Condition	Controller	success [%]	waypoints [%]	time [s]	crash [%]	episodes
nominal	PPO (ours)	99.4 ± 0.7	99.7 ± 0.3	6.58 ± 0.10	0.6 ± 0.7	500
nominal	Cascade PID	92.8 ± 2.3	97.5 ± 1.0	10.40 ± 0.16	2.2 ± 1.3	501

Table 6: Waypoint navigation (500 episodes). Time is measured over successful episodes only.

Reward iteration (documented). The first reward (proximity weight 0.2, collision −10, $\gamma = 0.99$) produced a policy that flies at the 3 m/s limit and accepts collisions (52.9 % success, 46.3 % collision): the proximity term contributed ≈ -0.005 per episode and the 2 s effective horizon favoured speed. Reward v2 (proximity 2.0, collision −20, $\gamma = 0.995$, everything else unchanged) was used for both the range and the depth policies. The classical potential-field baseline (same range ring) rarely collides but gets trapped in local minima.

Policy input / eval	Controller	success [%]	collision [%]	time [s]	min clear. [cm]	episodes
range ring (paired run)	Potential field	49.8 ± 4.4	5.2 ± 1.9	10.18 ± 0.41	70 ± 2	500
range ring, reward v1	PPO (ours)	52.9 ± 4.4	46.3 ± 4.4	3.67 ± 0.06	45 ± 2	501
range ring, reward v2	PPO (ours)	83.0 ± 3.3	16.6 ± 3.3	4.24 ± 0.05	53 ± 2	500
depth 64x48 (analytic, train dist.)	PPO (ours)	61.1 ± 4.3	36.9 ± 4.2	4.25 ± 0.05	50 ± 2	501
depth, zero-shot on RTX-rendered	PPO (ours)	66.7 ± 5.3	32.7 ± 5.3	4.22 ± 0.04	50 ± 3	300
range ring (independent run)	Potential field	39.3 ± 4.0	4.4 ± 1.7	9.86 ± 0.39	71 ± 2	570

Table 7: Obstacle navigation at full density (curriculum off). The depth policy (trained 900 iterations on analytic depth) is evaluated both on its training sensor and zero-shot on RTX-rendered depth. Minimum clearance is measured from the vehicle centre to the nearest cylinder surface (collision below 35 cm).

Findings (Table 7):

- **Reward v2** raised the range-ring policy from 52.9 to **83.0 %** success and cut collisions from 46.3 to 16.6 % at nearly the same speed (4.2 s to the goal).
- **Classical baseline.** The potential field rarely collides (4–5 %) but is trapped in local minima in 42–52 % of the episodes and is 2.4× slower. Two independent evaluation runs gave 49.8 and 39.3 % success; this spread exceeds the per-run CI because obstacle layouts are drawn from a random stream that the controllers consume differently, so episodes are not paired across runs and the CIs underestimate the between-run variance.
- **Depth policy.** With the forward-looking camera only (87° HFOV, no memory), the CNN policy reaches 61.1 % success on its training sensor, below the 360° range policy (83.0 %). A plausible but *unverified* explanation is that a memoryless policy cannot account for obstacles that have left the camera's field of view; a recurrent policy or a wider camera are the natural next experiments.
- **Analytic vs rendered depth.** Evaluated zero-shot with RTX-rendered depth, the depth policy achieves 66.7 % success / 32.7 % collisions, i.e. *no degradation* relative to the analytic depth it was

trained on (differences within the CIs). Together with the pixel-level agreement of Table 3, this justifies training depth policies with the fast analytic sensor (83 k vs 6.8 k env-steps/s, Table 9).

- **RGB / RGB-D.** The rendered RGB and RGB-D environments, image augmentation and the CNN actor-critic execute end to end (10-iteration PPO run with 64 environments). No RGB(-D) policy performance is claimed.

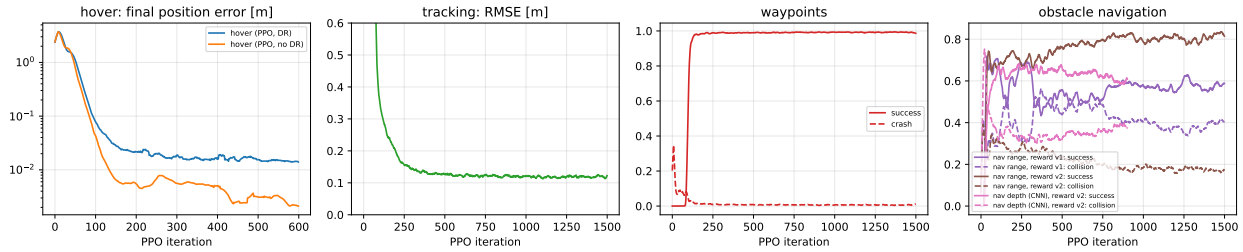


Figure 4: Training curves (episode-end metrics of the stochastic training policy, smoothed). The two hover curves are each measured on their own training distribution (the no-DR policy trains without noise) and are not directly comparable; see Table 8 for the controlled comparison. Navigation success is measured at the current curriculum density: the maximum obstacle count rises from 8 to 24, reached at iteration 112 (range v1), 124 (range v2) and 501 (depth v2); the early decline of the depth curve reflects the densifying forest. The depth run was stopped at iteration 900 (400 iterations at full density, success flat over the last 300) for compute-time reasons.

6.6 Domain-randomization ablation

An identical hover policy was trained without any randomization, sensor noise or latency (*w/o DR*) and compared with the default policy (*with DR*) under three evaluation conditions: *ideal* (the no-DR training distribution), *randomized* (the DR training distribution) and *harsh* (outside both training distributions: mass $\times[0.8, 1.25]$, motor time constants $\times[1.5, 2.5]$, thrust coefficients $\times[0.8, 1.0]$, sensor noise $\times[1.5, 2]$, 10–20 ms action delay, wind and gusts). The DR policy has lower position error in

Eval. condition	Policy	success [%]	crash [%]	final pos. err [cm]	final yaw err [deg]	episodes
ideal	PPO with DR	100.0 ± 0.0	0.0 ± 0.0	0.22 ± 0.00	3.7 ± 0.1	1000
ideal	PPO w/o DR	100.0 ± 0.0	0.0 ± 0.0	0.28 ± 0.00	9.8 ± 0.2	1000
randomized	PPO with DR	99.9 ± 0.2	0.1 ± 0.2	1.38 ± 0.26	4.4 ± 0.2	1000
randomized	PPO w/o DR	99.9 ± 0.2	0.1 ± 0.2	2.05 ± 0.29	10.1 ± 0.2	1000
harsh	PPO with DR	78.5 ± 2.5	3.9 ± 1.2	16.37 ± 2.39	19.9 ± 2.4	1011
harsh	PPO w/o DR	77.2 ± 2.6	4.9 ± 1.3	18.58 ± 2.46	23.1 ± 2.0	1009

Table 8: DR ablation on the hover task (1000 episodes per row, identical seeds).

every condition (e.g. 1.38 vs 2.05 cm under the training randomization, 16.4 vs 18.6 cm under harsh conditions) and fewer crashes under harsh conditions (3.9 vs 4.9 %), but the success-rate difference under harsh conditions (78.5 vs 77.2 %) is within the 95 % confidence interval. We therefore do *not* claim a significant robustness gain from DR for this task. A plausible explanation is the CTBR interface: the onboard rate loop closes around the (real) gyro and absorbs much of the inertial and actuator variation, a robustness property reported for CTBR policies in [3]. The yaw error of the DR policy is 2–3× lower in all conditions; with one seed per arm this may be a training-run effect rather than a DR effect. Multi-seed ablations, and ablations with the direct motor interface where DR is expected to matter more, are planned (Section 10).

6.7 M6: export and deployment pipeline

The exported ONNX graph (normalizer + actor + clamp) matches PyTorch to 3.6×10^{-7}. In the deployment-in-the-loop test the simulator is flown by the ONNX policy whose observation is rebuilt every step by the NumPy `ObservationBuilder` from onboard quantities only. Over 512 hover episodes the maximum observation error is 9.5×10^{-7}, the maximum action error 1.0×10^{-6}, and the success rate (99.8 %) and final error (1.29 cm) equal those of the PyTorch policy. Frame conversions to PX4 (NED/FRD) are unit-tested, including the consistency of a physical body vector through both quaternion conventions.

6.8 Off-policy baseline: SAC

To support algorithms beyond PPO, an off-policy Soft Actor-Critic trainer (skrl 1.4.3) runs on the same environments (`scripts/rl/train_sac.py`). On the hover task with 256 environments and 30 000 vectorized steps (7.7 M transitions, 30 k gradient updates, 21 min) the episode return rose monotonically from −11.2 to +1.7 (still increasing; PPO reaches ≈ 11 on the same reward). The deterministic policy never crashed but ended 1.13 m from the target on average (0 % success over 1024 episodes). SAC is therefore *executing and learning* but *not competitive* within this budget; a higher update-to-data ratio and tuning are required.

6.9 M7: throughput, memory and containerization

Table 9 lists environment-stepping throughput (no learning) on the idle RTX 5060 (8 GB); VRAM is the whole process including Isaac Sim. State-based tasks scale almost linearly up to 8192 environments (0.44 M env-steps/s, 1.76 M physics steps/s at 200 Hz). Analytic depth (Warp kernel, 3072 rays per environment) costs only ≈1.6× relative to the state task at 2048 environments, whereas RTX-rendered cameras are limited to a few thousand env-steps/s. End-to-end training throughput (with PPO updates) was ≈190 k (hover, 4096 envs), 117 k (range navigation) and 16 k env-steps/s (depth CNN, 2048 envs). The Docker image (`docker/Dockerfile`, based on `nvcr.io/nvidia/isaac-lab:2.3.2`)

Task	envs	k env-steps/s	ms / step	VRAM [GB]
`Hover-v0`	256	16.1	15.92	2.27
`Hover-v0`	1024	63.0	16.26	2.96
`Hover-v0`	4096	237.0	17.29	3.72
`Hover-v0`	8192	438.9	18.66	3.86
`Nav-Depth-v0`	256	14.5	17.67	2.33
`Nav-Depth-v0`	1024	52.5	19.51	3.24
`Nav-Depth-v0`	2048	83.1	24.65	4.41
`Nav-Depth-Render-v0`	64	2.1	29.85	4.17
`Nav-Depth-Render-v0`	128	3.8	33.77	4.85
`Nav-Depth-Render-v0`	256	6.8	37.79	5.60
`Nav-RGBD-v0`	64	2.1	30.66	4.19
`Nav-RGBD-v0`	128	3.8	33.28	4.87

Table 9: Environment throughput and process VRAM (`scripts/tools/benchmark.py`).

builds, and the containerized test service passes all unit tests and the Isaac Sim GPU smoke test. Interactive GUI visualization (chase camera, target/reference markers) and headless MP4 recording of trained policies were verified after the patch of Section 6.3.1.

7 PX4 SITL validation and sim-to-sim analysis

The exported policies were flown as PX4 offboard controllers against PX4 v1.16.2 software-in-the-loop (SITL) with the Gazebo Harmonic `x500` model, whose rotor, motor and inertia parameters equal the quadsim X500 preset. SITL replaces quadsim's rigid-body engine (PhysX → gz-physics), its onboard

controllers (simulated rate PID and velocity cascade → PX4 `mc_rate_control`, `mc_pos_control` and the control allocator) and its estimator model (noise model → EKF2 fusing simulated IMU, GNSS, barometer and magnetometer), and adds the real deployment software path (ROS 2 node, uXRCE-DDS, PX4 time synchronization). Differences between the two simulators therefore expose the assumptions of the training environment that a real flight would also violate. Every experiment is scripted and re-runnable (`ros2/px4_sitl/setup_px4_sitl.sh`, `scripts/sitl/`); all reported numbers are computed by the same unit-tested functions (`quadsim.deploy.closed_loop_metrics`) for quadsim and SITL.

7.1 Setup and deployment pipeline

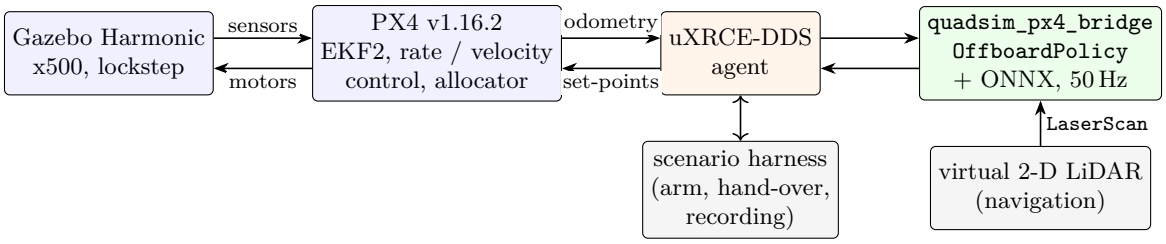


Figure 5: SITL loop. The bridge receives `VehicleOdometry` (EKF2, NED/FRD) and publishes `OffboardControlMode` plus `VehicleRatesSetpoint` (CTBR policies) or `TrajectorySetpoint` (velocity policies). The harness takes off with PX4, calibrates the thrust map, hands over to the policy, applies the task and records odometry, simulator ground truth and commands in one time base.

PX4 runs its EKF2 and controllers unmodified with airframe defaults, except `NAV_DLL_ACT`=0 (no ground station) and `EKF2_GPS_DELAY`=0 (Section 7.3). The only change to PX4 is SITL instrumentation that exports the gz ground truth over DDS for evaluation. The bridge's per-step logic is the ROS-free class `quadsim.deploy.offboard.OffboardPolicy`: it rebuilds the training observation of each task (hover goal; tracking reference $t + \{0, \dots, 1\}$ s generated onboard; waypoint sequence with the training switching rule; navigation goal plus the 32-ray range ring resampled from any `LaserScan`) and is checked against the simulator in closed loop (observation error $\leq 7.5 \times 10^{-6}$ for all four tasks, ONNX vs PyTorch success within 3 points; generalized M6 test). A scenario starts with a PX4 take-off and Hold; the goal / reference / course is defined from the current pose and the policy takes over in OFFBOARD; after the task PX4 takes back control and lands.

Thrust map. PX4 expects a normalized thrust $u \in [0, 1]$. For gz `x500` the ESC maps u to $\omega = \omega_{\min} + (\omega_{\max} - \omega_{\min})u$ and the thrust is $T = 4k_f\omega^2$, i.e. $T/T_{\max} = c(u) = (0.15 + 0.85u)^2$. The documented calibration (linear map $u = s\,C/T_{\max}$ with $s = u_h/(mg/T_{\max})$ from PX4's hover-thrust estimate u_h) is exact only at hover; there the true local gain exceeds the linear map's by

$$\frac{c'(u_h)}{c(u_h)/u_h} = \frac{2 \cdot 0.85\,u_h}{0.15 + 0.85\,u_h} = 1.61 \qquad (u_h = 0.7295). \tag{7}$$

The bridge therefore also supports an exact curve map $u = c^{-1}(s\,C/T_{\max})$ with $s = c(u_h)/(mg/T_{\max})$ (`ThrustCurve`); the calibrated $s = 1.0016$ shows that the gz model and the quadsim preset agree on hover thrust within 0.2 %.

Identification tools. Command→response dynamics are identified from closed-loop flight data by an output-error fit of a first-order-plus-dead-time model $Y(s)/U(s) = Ke^{-Ls}/(\tau s + 1)$ (grid over L and τ, least squares for K, zero-order-held commands); estimator lag by the shift minimizing $\mathrm{E}\|\hat{x}(t) - x(t - L)\|^2$; limit cycles by the RMS and Welch-spectrum peak of body rate, collective command and vertical velocity. All are unit-tested on synthetic signals with known parameters.

7.2 Defects found by SITL

- **Bridge never commanded.** PX4 1.16 versions some DDS topics (`VehicleStatus`→ `/fmu/out/vehicle_status_v1`); the original node subscribed to the unversioned name, never

saw OFFBOARD and sent no set-point. Reproduced: PX4 accepted OFFBOARD for 8 s without receiving a single set-point and the vehicle drifted 3.5 m on the last attitude-controller command. Topic names are now derived from the message versions.

- **Failsafe could never trigger.** PX4 detects offboard loss only from the `OffboardControlMode` heartbeat (`COM_OF_LOSS_T`); the node published it before its stale-odometry and geofence checks. The heartbeat is now sent only while the node can command; verified: after a geofence violation PX4 left OFFBOARD after 1.03 s and, without RC, landed.
- Packaging (`setup.cfg`), a *last-action* input containing never-applied commands before the hand-over, NaN/frame checks of the odometry.
- **Measurement artefacts.** PX4's gz ground-truth velocity and "angular velocity" are backward differences (the latter of Euler angles); truth rates are recomputed by central differences of the ground-truth quaternions. rclpy's multi-threaded executor dropped ≈35 % of the messages in the recorder (PX4 itself received every set-point).

7.3 Estimator

Table 10 summarizes EKF2's error against the gz ground truth. In hover it is dominated by a per-flight bias (2–4 cm) and a slow, correlated component (std 1.4 cm, correlation time 2–3 s); the white part is ≈1 mm and the attitude error ≈0.2 mrad. quadsim's v0 odometry model is the opposite: mostly white (1 cm position, 3 cm/s velocity, 10 mrad attitude). In motion, EKF2 led the truth by 0.11 s: its default GNSS delay compensation (`EKF2_GPS_DELAY`=110 ms, correct for real receivers) assumes a latency the gz GNSS does not have. Matching it (0 ms) removed the lead and halved the truth-referenced tracking error (14→7 cm). These statistics define the EKF-like model of the v1 training distribution.

Quantity	PX4 EKF2 in SITL
Per-flight position bias $\lvert\bar{e}_p\rvert$ (x / y / z) [cm]	1.8 / 1.8 / 3.9
Slow position error (<0.5 Hz) std [cm]	1.4 / 1.4 / 1.5
Slow position error correlation time [s]	1.7 / 1.9 / 3.2
Fast position error std [mm]	1.3 / 1.5 / 1.1
Velocity error std [cm/s]	0.94 / 1.05 / 0.41
Velocity error correlation time [s]	0.81 / 0.35 / 2.14
Roll / pitch error std [mrad]	0.19 / 0.20
Circle flight, `EKF2_GPS_DELAY`=110 ms: lag x / y [ms], RMS error [cm]	-110 / -99, 12.0
Circle flight, `EKF2_GPS_DELAY`=0 ms: lag x / y [ms], RMS error [cm]	-3 / -1, 6.3

Table 10: PX4 EKF2 error vs gz ground truth (`scripts/sitl/estimator_stats.py`, `docs/results/sitl_estimator.json`).

7.4 Hover and set-point steps

The scenario of Table 11 holds the hand-over pose for 30 s (RMS over 10–30 s), then steps the goal by 1 m in x and in z; quadsim flies the same schedule with 64 environments starting at the goal.

Findings: (i) *Control performance transfers*: referenced to the estimate, SITL holds the pose to 0.8 cm to 1.0 cm RMS (quadsim 1.9 cm) and the z step rises and settles as in quadsim; the smoother EKF2 estimate produces much quieter commands than quadsim's white-noise odometry. (ii) With PX4's default `EKF2_GPS_DELAY` the x step settled slower (1.86 s, the position dipping back to ≈0.9 m); with the GNSS delay matched to gz the same policy settles in 1.01 s to 1.04 s (quadsim 1.07 s) – the dip was an estimator artefact of the SITL configuration, not a control difference. (iii) *Heave limit cycle*: with the linear thrust map, 3 of 5 runs developed a persistent 3.75 Hz to 4 Hz vertical oscillation (collective std 0.30–0.36, vertical velocity 0.14–0.17 m/s RMS) from the hand-over on; with the curve map 0 of 4 runs did. Quadsim shows the same lightly damped mode as noise-driven heave;

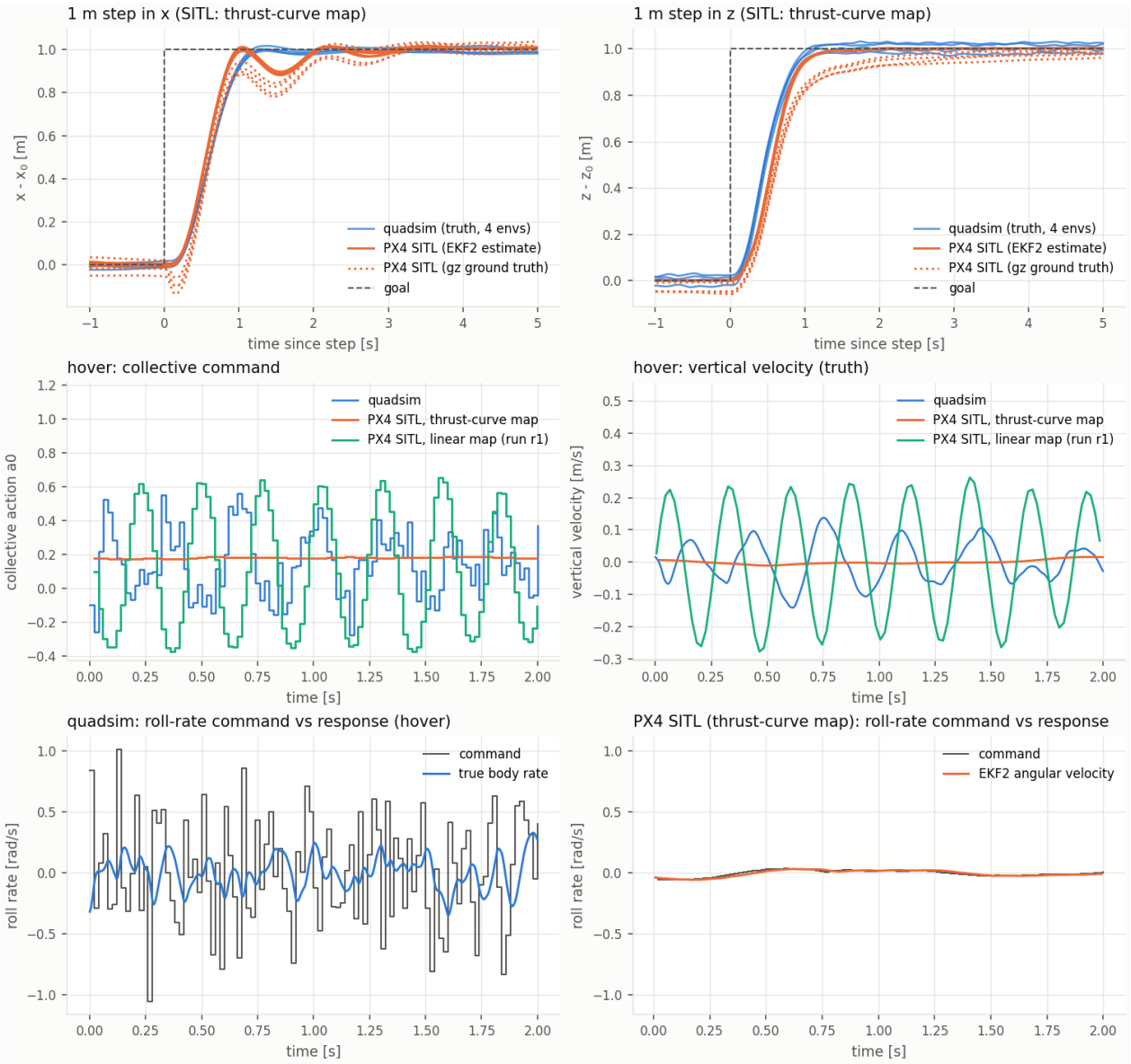


Figure 6: Top: 1 m steps (quadsim truth, SITL estimate and truth). Middle: hover collective command and vertical velocity – with the linear thrust map PX4 SITL shows a 4 Hz heave limit cycle that the thrust-curve map removes. Bottom: roll-rate command and response (quadsim commands chatter because of the white odometry noise).

Metric	quadsim nominal	SITL linear map (3)	SITL curve map (4)	SITL linear + geometry fix (2)
Hover RMS error, truth [cm]	1.59 ± 0.46	4.12 ± 1.94	6.00 ± 1.11	4.71 ± 1.06
Hover RMS error w.r.t. estimate [cm]	1.88 ± 0.02	1.02 ± 0.13	0.83 ± 0.04	1.01 ± 0.22
Estimate vs truth, hover RMS [cm]	2.28 ± 0.31	3.84 ± 2.08	5.81 ± 1.19	4.51 ± 1.08
x step w.r.t. estimate: rise 10–90 % [s]	0.65 ± 0.01	0.54 ± 0.01	0.55 ± 0.01	0.54 ± 0.00
x step w.r.t. estimate: settling (5 cm) [s]	1.07 ± 0.02	1.84 ± 0.02	1.86 ± 0.02	1.86 ± 0.02
z step w.r.t. estimate: rise 10–90 % [s]	0.63 ± 0.01	0.64 ± 0.01	0.64 ± 0.01	0.62 ± 0.02
z step w.r.t. estimate: settling (5 cm) [s]	1.00 ± 0.03	1.07 ± 0.05	1.02 ± 0.02	1.07 ± 0.03
Rate cmd→rate: delay roll [ms]	13 ± 4	23 ± 1	31 ± 1	15 ± 5
Rate cmd→rate: delay pitch [ms]	13 ± 4	13 ± 2	10 ± 1	10 ± 2
Rate cmd→rate: τ roll [ms]	53 ± 2	47 ± 2	38	38 ± 3
Rate cmd→rate: τ pitch [ms]	52 ± 0	39 ± 1	42 ± 1	41 ± 3
Hover collective a_0 std	0.174 ± 0.018	0.218 ± 0.139	0.005 ± 0.000	0.178 ± 0.174
Hover vertical velocity RMS, truth [cm/s]	5.4 ± 1.1	10.1 ± 6.2	1.0 ± 0.0	8.8 ± 7.8
Odometry age at policy step, median [ms]	–	11.9 ± 0.9	8.0 ± 1.8	12.5 ± 3.2

Table 11: Hover policy (v0, seed 42), quadsim vs PX4 SITL (number of SITL runs in brackets; ± = std over envs / runs). "w.r.t. estimate" measures what the policy regulates; truth-referenced numbers include EKF2's error. These SITL runs used PX4's default `EKF2_GPS_DELAY`=110 ms, which slows the x step (see (ii)); later runs use 0 ms. Source: `docs/results/sitl_comparison.json`.

the 1.6× local thrust gain of the linear map turns it into a limit cycle. (iv) *Rate loop*: PX4's pitch response matches quadsim's delay (8–14 vs 13 ms) with a smaller time constant (38–44 vs 53 ms); roll is slower (22–32 ms) because the x500 airframe's allocation geometry (`CA_ROTOR*`=0.13/0.20–0.22 m) differs from the gz rotor positions (±0.174 m); setting the true geometry reduced the roll delay by 4–12 ms under matched conditions (2 runs). Fitted delays depend on the excitation by ±10 ms (closed-loop identification); with PX4's backward-differenced ground-truth rates all SITL delays appear ≈10 ms longer. (v) PX4's hover-thrust estimate is 0.7295 (default `MPC_THR_HOVER` 0.60); without calibration ($s = 1$, −19 % thrust) the policy still flies but holds 1.3 cm low. (vi) Odometry is 5–16 ms old at the policy step (median per run, max 23 ms) – at and above the trained 0–10 ms.

7.5 Training distributions v1 / v2 and multi-seed comparison

The findings above define the v1 distribution (Section 5.3). Hover policies were trained with three seeds on v1 and compared with three seeds of v0 (seed 42 plus two new seeds) and, after the v1 result, three seeds of a corrected distribution v2, in quadsim and in SITL with `EKF2_GPS_DELAY`=0 (Table 12). **v1 is a negative result.** Making the training estimate noise realistic (correlated, little white noise) and randomizing the thrust-map gain up to 1.7 produced policies with *more* collective chatter: 4 of 6 SITL flights with the linear map developed the heave limit cycle (v0: 1 of 6), one even with the curve map, the SITL hover error doubled (2.2 vs 1.0 cm), and success under the harsh out-of-distribution condition dropped from 78 to 67 %, while in-distribution performance was unchanged. **v2** keeps v1's correlated estimate errors, latencies and gain randomization, restores v0's white-noise level and adds a collective-rate penalty $-w\,(a_{0,k} - a_{0,k-1})^2 \Delta t$ ($w = 1$). It halves the collective chatter in quadsim (std 0.08 vs 0.15), keeps v0's hover precision, and *none* of its 9 SITL flights (3 seeds, linear and curve maps) showed the limit cycle. The price is a less damped vertical step (overshoot 7–20 % for two seeds vs 1–2 % for v0), lower harsh-condition success (69 %) and a larger yaw error. No training distribution dominates: v0 with the thrust-curve map is the best-validated deployment configuration, v2 the more robust choice when only a linear thrust map is available. Across all nine policies, the collective chatter in the quadsim thrust-gain scenario ranks the policies as SITL does, which makes it a cheap pre-deployment screen.

7.6 Trajectory tracking and waypoints

Table 13 compares the tracking policy on a circle ($r = 1$ m, 0.2 Hz, 1.26 m/s) and a figure-8 ($r = 1.5$ m, 0.15 Hz) and the waypoint policy on a fixed 5-waypoint course, all with the thrust-curve map. Referenced to the estimate, tracking in SITL equals quadsim (circle 2.2 vs 2.5 cm, figure-8 3.4 vs 3.0 cm RMSE); the truth-referenced error is 3–4× larger because of EKF2's correlated error. The waypoint course was completed in every SITL run (policy's own estimate-based switching) but took

Metric	v0 (3 seeds)	v1 (3 seeds)	v2 (3 seeds)
quadsim, v0 distribution: success [%]	99.9 ± 0.0	99.8 ± 0.3	99.2 ± 0.6
quadsim, v0 distribution: final pos. error [cm]	1.33 ± 0.09	1.88 ± 0.37	1.52 ± 0.20
quadsim, v1 distribution: success [%]	72.9 ± 2.6	75.6 ± 5.4	69.3 ± 5.2
quadsim, v1 distribution: final pos. error [cm]	6.74 ± 0.09	6.78 ± 0.08	6.65 ± 0.11
quadsim, harsh OOD: success [%]	78.1 ± 1.3	66.5 ± 4.3	68.5 ± 2.4
quadsim, harsh OOD: crash [%]	3.9 ± 0.6	3.9 ± 0.4	4.3 ± 0.3
scenario $g = 1.0$: hover RMS w.r.t. estimate [cm]	1.87 ± 0.00	2.46 ± 0.32	1.95 ± 0.05
scenario $g = 1.0$: collective std	0.149 ± 0.016	0.211 ± 0.049	0.079 ± 0.003
scenario $g = 1.6$: collective std	0.209 ± 0.026	0.286 ± 0.079	0.112 ± 0.007
scenario $g = 1.6$: vertical velocity RMS [cm/s]	8.7 ± 1.0	9.0 ± 2.3	8.6 ± 0.8
SITL linear map: runs with limit cycle	1 / 6	4 / 6	0 / 6
SITL curve map: runs with limit cycle	–	1 / 3	0 / 3
SITL linear map: collective std	0.061 ± 0.080	0.310 ± 0.171	0.062 ± 0.016
SITL linear map: hover RMS w.r.t. estimate [cm]	1.00 ± 0.10	2.17 ± 0.27	1.26 ± 0.23
SITL linear map: x step settling [s]	1.02 ± 0.02	1.05 ± 0.01	0.99 ± 0.05
SITL curve map: collective std	–	0.164 ± 0.142	0.031 ± 0.025
SITL curve map: hover RMS w.r.t. estimate [cm]	–	2.16 ± 0.74	1.29 ± 0.21

Table 12: Hover policies trained on v0, v1 and v2, mean ± std over 3 seeds. Quadsim: 1000 episodes per seed and condition; scenario: hover + steps with nominal dynamics and a thrust-map gain g (1.6 = linear map on the gz x500). SITL: limit-cycle runs = collective std > 0.1 in hover. Source: `docs/results/hover_v1.json`.

6.7 s instead of 5.1 s; at up to 7 m/s EKF2's error reaches 30 cm, so on truth some waypoints were passed just outside the 0.3 m radius. Removing quadsim's body drag did not change its time (5.13 s); in SITL neither the true allocation geometry (6.29 s) nor roll/pitch air mode (`MC_AIRMODE`=1, 6.45 s / 6.20 s) changed it beyond the run-to-run spread. The cause remains open (candidates: EKF2's error at 7 m/s acting on the estimate-based switching and approach geometry, motor saturation at full thrust).

Task	Metric	quadsim	PX4 SITL
Tracking, circle (r 1 m, 0.2 Hz)	RMSE truth [cm]	1.8 ± 0.4	6.3 ± 0.7
	RMSE w.r.t. estimate [cm]	2.5 ± 0.1	2.2 ± 0.2
Tracking, figure-8 (r 1.5 m, 0.15 Hz)	RMSE truth [cm]	2.7 ± 0.2	9.2 ± 0.4
	RMSE w.r.t. estimate [cm]	3.0 ± 0.1	3.4 ± 0.4
Waypoints (5)	course completed	1.00	1.00
	time to finish [s]	5.13 ± 0.02	6.74 ± 0.27
	estimate error during course, RMS [cm]	–	15.2 ± 0.7

Table 13: Tracking and waypoint policies (v0), quadsim (32 envs) vs PX4 SITL (2 runs each); RMSE after the first 2 s. Source: `docs/results/sitl_tasks.json`.

7.7 Obstacle navigation through the velocity interface

The range-ring navigation policy commands velocities, which PX4's position controller tracks. Obstacles are virtual in SITL: a 2-D LiDAR node ray-casts the training ring against the cylinders of a layout from the ground-truth pose (identical to the training caster, unit-tested), and collisions are evaluated from the ground truth with the training criterion. Six layouts drawn from the full-density training distribution were flown once in SITL and by 32 quadsim environments each (Table 14). With PX4's default velocity controller only 2 of 6 SITL flights reached the goal, against 87 % quadsim success on the same layouts. Identifying both velocity loops from the flight data explains the gap: PX4's lateral velocity response has $L = 0.21$ s, $\tau = 0.52$ s (VAF 0.78, 6 flights) against $L = 0.14$ s,

$\tau = 0.22\,\mathrm{s}$ (VAF 0.91) for quadsim's simulated cascade – PX4's default horizontal velocity gain (`MPC_XY_VEL_P_ACC`=1.8) is lower than the simulated one (3.0), so avoidance manoeuvres arrive late. Configuring PX4's velocity controller like the training cascade (`MPC_XY_VEL_P_ACC`=3.0, D=0, `MPC_Z_VEL_I_ACC`=1.0, tilt limit 0.6 rad) reduced the lateral time constant to 0.25 s (quadsim 0.22 s) and doubled the SITL success to 4 of 6 layouts; the remaining failures are layout 1 (hard in quadsim as well, 38 % success) and layout 4. The residual lateral delay (≈0.21 s vs 0.14 s) comes from PX4's attitude and rate chain and is not represented in training. With one SITL flight per layout these rates are indicative only; the lesson for deployment is that velocity-interface policies must be trained with the target autopilot's velocity-loop dynamics (identified, or randomized over a range that contains them).

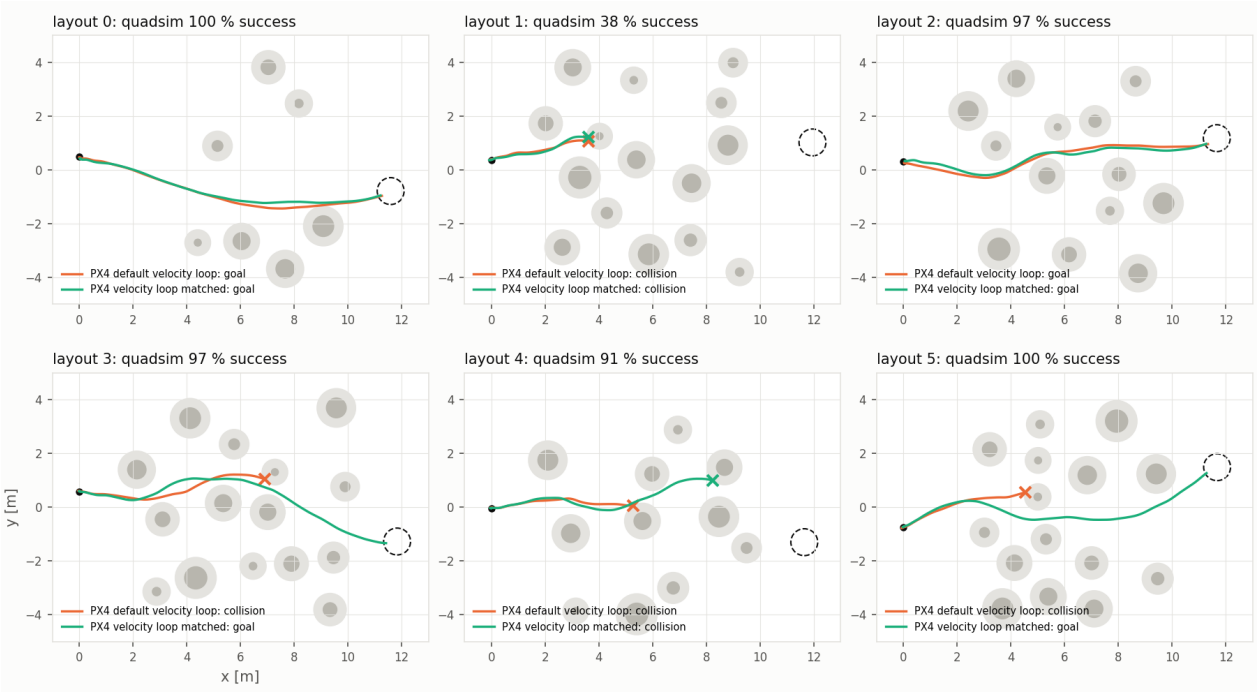


Figure 7: PX4 SITL navigation flights in layout coordinates (grey: obstacles, light grey: collision margin of the 0.35 m vehicle radius; dashed: goal region). With PX4's default velocity controller (orange) avoidance arrives late and the vehicle cuts into obstacle margins; with the velocity loop matched to the training cascade (green) 4 of 6 flights reach the goal.

Layout	obstacles	quadsim success / collision [%]	PX4 default velocity loop	PX4 velocity loop matched to quadsim
0	7	100 / 0	goal	goal
1	15	38 / 62	collision	collision
2	13	97 / 3	goal	goal
3	15	97 / 3	collision	goal
4	11	91 / 6	collision	collision
5	15	100 / 0	collision	goal

Table 14: Navigation layouts: quadsim success / collision rate (32 envs per layout) and the outcome of the PX4 SITL flight with PX4's default and with a quadsim-matched velocity controller. Source: `docs/results/sitl_tasks.json`.

7.8 Summary of the cross-check

- All CTBR policies (hover, tracking, waypoints) fly PX4 SITL as offboard controllers; at the level the policy controls (estimate-referenced) their performance matches quadsim; the bridge's safety gating is verified.

- Sim-to-sim gaps were found and quantified, each with an identified cause: EKF2's correlated error (training model too white), GNSS delay mis-configuration in SITL, the thrust-map nonlinearity (heave limit cycle; fixed by the curve map), PX4's roll/pitch-asymmetric rate loop (allocation geometry), up to 20 ms observation age, and – the largest – PX4's slower velocity loop for the velocity-interface navigation policy.
- Training on the SITL-identified distribution is not automatically better: v1 worsened the heave mode, v2 fixed it with trade-offs (multi-seed evidence, Table 12); the deployment-side fix (thrust-curve map) is the most robust.
- Not done: hardware flights, depth/RGB policies in SITL (needs a rendered camera in gz), retraining the navigation policy with a randomized velocity loop, tracking/waypoint policies on the new distributions.

8 From platform to research framework

The platform of the previous sections was built around a fixed set of tasks. An audit of that code base for reuse by other researchers found four structural limits: (i) the action interfaces were branches of one controller function, so a new interface required editing the base environment; (ii) vehicles were two hard-coded Python presets (quadrotors only for direct motor control); (iii) observations, rewards, terminations and randomizations could only be extended by subclassing a task; (iv) training was tied to one algorithm library, with a separate, non-competitive SAC script, and there were no standard benchmark conditions, no constrained or multi-agent tasks and no imitation-learning data path. This section describes the framework that replaced these limits without rebuilding the validated core, and the experiments that validate it. Architecture details are in `docs/FRAMEWORK.md`.

8.1 Extension points

Every replaceable component type has a name registry; components register themselves on import, research code registers its own (from any module or file passed as `-plugins` to every script) and selects them by name in the configuration or on the command line (Table 15). Configuration-level extensions do not need a registry: each task configuration carries *terms* with the Isaac Lab signature `func(env, **params)` (Table 16).

Table 15: Registries (`quadsim.registry`).

Registry	Selected by	Built-in components
`VEHICLES`	`dynamics.preset` (name or YAML/JSON file)	x500, crazyflie, racer5, hexa_x
`ACTION_INTERFACES`	`control.mode`	motor, ctbr, ctatt, velocity, position
`ALGORITHMS`	`-algo`	ppo (rsl_rl), sac, td3, bc, vla
`BASELINES`	`-controllers`	cascade, geometric, potential_field, swarm_potential, langnav_expert
`BENCHMARKS`	`benchmark/run.py -suite`	hover, track, waypoint, nav_range, race, swarm, langnav

Sensor terms. A term h with period P and latency D (both in policy steps) produces $o_t = h(s_{P\lfloor (t-D)/P \rfloor}) + \eta$, $\eta \sim \mathcal{N}(0, \sigma^2 I)$, with the history refilled at every episode start. The same mechanism implements a 5 Hz GNSS fix with 100 ms latency, a downward rangefinder, a 3-D LiDAR ray-casting the task's obstacle primitives, or action/gyro histories for implicit adaptation.

Vehicles. A vehicle file may inherit from a preset, generate a symmetric n-rotor frame (4-rotor X frames use the PX4 Quad-X numbering), derive k_f from the thrust-to-weight ratio and estimate the inertia from a box body and point-mass motors. Vehicles are rejected if the inertia violates the

Table 16: Configurable MDP terms (`cfg.terms`).

Term	Semantics
observation	appended to the deployable policy observation (or the critic's, if privileged) with sampling period P, latency D, white noise, scale, clip and an optional history of H samples
reward	$w\, f(s_t)\, \Delta t$, logged per term
termination	failure, or truncation (bootstrapped)
cost	constraint cost rate $c(s_t) \geq 0$; episode cost $J_c = \sum_t c\, \Delta t$ (seconds of violation for an indicator)
event	start-up / reset / interval randomization or perturbation
curriculum	called before resets with the finished episodes' metrics (e.g. raise the initial tilt when success $> 80\,\%$)

triangle inequality, the thrust-to-weight ratio is ≤ 1.05 or the allocation matrix $A \in \mathbb{R}^{4\times n}$ (collective, roll, pitch, yaw) has rank < 4. The USD body, PhysX mass properties, motor and drag model, pseudo-inverse mixer with PX4-style desaturation, hover action and the motor-level action dimension follow automatically. An Isaac Lab `MultirotorCfg` (allocation matrix with rows $[F_x, F_y, F_z, M_x, M_y, M_z]$, thrust $c\,\mathrm{rps}^2$) converts with $y_i = A_{3i}$, $x_i = -A_{4i}$, $s_i = -\mathrm{sign} A_{5i}$, $k_f = c/(2\pi)^2$.

Constrained RL. With cost terms and `safety.lagrangian`, the environment optimizes the Lagrangian of $\max_\pi J_r(\pi)$ s.t. $J_c(\pi) \leq d$ by returning $\tilde{r}_t = (r_t - \lambda c_t)/(1+\lambda)$ to *any* RL algorithm and updating λ with the PID rule of Stooke et al. [12]: $e_k = \bar{J}_c - d$, $I_k = \max(0, I_{k-1} + K_i e_k)$, $\lambda_k = \min(\lambda_{\max}, \max(0, K_p e_k + I_k + K_d \max(0, \bar{J}_{c,k} - \bar{J}_{c,k-1})))$, where $\bar{J}_c$ is the mean cost of the episodes finished since the last update (every rollout). This is the reward-penalty form (no separate cost critic).

Multi-agent RL. M consecutive environment rows form a team sharing a local frame, an episode clock and resets; each agent is one row. Any single-agent algorithm therefore trains a parameter-shared decentralized policy $\pi(a^i \mid o^i)$, while the critic receives the centralized state (true relative states and goals of all team members), i.e. a MAPPO-style [13] centralized-training, decentralized-execution scheme. A crash terminates the crashed agent and truncates its teammates; inter-agent collisions are analytic.

Off-policy learning. The original skrl SAC [9] reached 0 % hover success. Two likely causes were addressed together (not ablated separately): the tasks' rewards are Δt-scaled (about 0.02 per step), so the entropy bonus dominated, and the critic only saw the policy observation; the update-to-data ratio was also low (1/256). The framework's SAC/TD3 use asymmetric twin critics with LayerNorm on the privileged observation, running observation normalization, rewards in per-second units ($r/\Delta t$), 1024 parallel environments, batches of 16 384 and 4 gradient updates per environment step, following the many-environment recipe of FastTD3 [11]; TD3 [10] shares the implementation. Time-outs bootstrap from the current state (as in rsl_rl's PPO).

Imitation learning and VLA. Demonstrations are episode-structured HDF5 files (policy and critic observations, camera frames, actions, rewards, ground truth, estimate, instruction text, success, metrics, configuration, observation layout, tokenizer). Collection gives every environment the same episode quota, because stopping at a global count discards the episodes still running, which are the long ones. Behaviour cloning minimizes $\mathbb{E}\|\pi_\theta(o) - a^\star\|^2$; instruction tokens are embedded with a bag-of-words encoder that ignores padding and out-of-vocabulary words, and images are encoded by a CNN whose last feature map is FiLM-modulated [17] by the instruction embedding. DAgger [15] alternates rollouts of the learner (labelled by the expert) with retraining on the aggregated data. External VLA models plug in through an adapter $(\text{images}, \text{instructions}, \text{proprioception}) \mapsto a$; datasets

export to the LeRobot [19] v3.0 format and are verified by reloading them with LeRobot's own loader.

Benchmarks. A suite fixes a task, a set of conditions labelled in-distribution (training distribution) or out-of-distribution (wind and pushes, harsh dynamics, sensor noise $\times 2$–3, 20 ms to 30 ms latency, unseen layouts or phrasings), metrics and seeds. Every controller is evaluated per condition in a fresh simulator process with the same seed; the report gives each metric with its 95 % CI and the generalization gap id $-$ $\overline{\text{ood}}$ of the primary metric.

Resources. The default number of environments is lowered automatically when the free GPU memory would not fit it (memory model calibrated on the RTX 5060 measurements; never raised unless requested).

8.2 Validation of the framework

Regression. After moving the action interfaces into classes, the vehicles into a registry and adding the term machinery, the reference evaluations (hover, tracking, waypoint, range navigation; PPO policies and the cascade, SE(3) and potential-field baselines) were re-run: all 59 metrics are bit-identical to the pre-refactor run.

Unit and example tests. 124 CPU unit tests (registries, vehicle files and frames, allocation rank, Isaac Lab conversion, hover action of every interface on every vehicle, closed-loop position control of a hexarotor and a racer on the reference integrator, sensor-term rate/latency/history, the PID multiplier, the squashed-Gaussian log-likelihood, replay buffer, hardware model, tokenizer, datasets, BC language grounding) and an example check that runs every extension example in the simulator (Table 23).

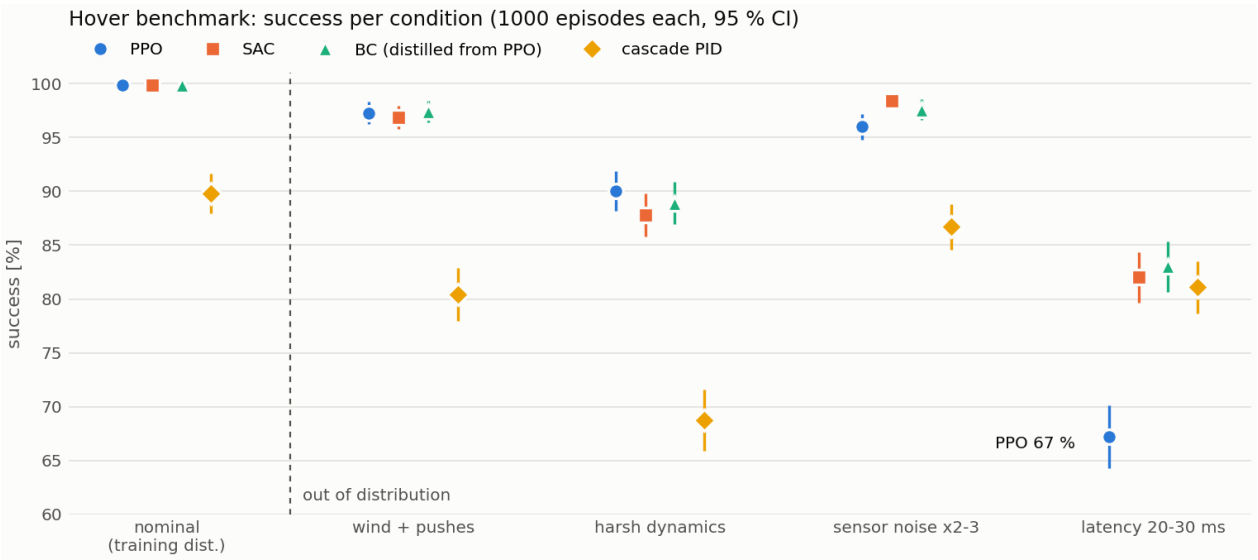


Figure 8: Hover benchmark suite: success per evaluation condition for PPO, the framework's SAC, a BC student distilled from the PPO policy and the cascade baseline (1000 episodes per point, same seed, 95 % CI; values in Table 17).

8.3 Findings

- **Algorithms (Tables 17 and 18 and Fig. 8).** With the changes above SAC matches PPO's in-distribution hover success (99.9 %) in 21 instead of 8 minutes, with a slightly larger position error and a much larger yaw error (its entropy term makes it indifferent to the weakly rewarded

Table 17: Hover benchmark suite: success per condition (1000 episodes each, same seed, fresh process per controller and condition) and generalization gap id−mean(ood) in points. Source: `docs/results/framework/bench_hover_algos.json`.

Controller	nominal (id)	wind_push	harsh_dynamics	sensor_noise	latency	gap
ppo	99.9 %	97.3 %	90.1 %	96.0 %	67.2 %	+12.3
sac	99.9 %	96.9 %	87.8 %	98.4 %	82.0 %	+8.6
bc_distilled	99.9 %	97.4 %	88.9 %	97.6 %	83.0 %	+8.2
cascade	89.8 %	80.4 %	68.7 %	86.7 %	81.1 %	+10.6

Table 18: Learning algorithms on hover (nominal condition, 1000 episodes). Position error in cm, yaw error in rad. Source: `offpolicy_hover.json`.

Algorithm	Success	Pos. err [cm]	Yaw err [rad]	Train [min]
PPO (rsl_rl), 4096 envs, 600 it.	99.9 %	1.38 ± 0.26	0.08 ± 0.00	8
SAC (quadsim), 1024 envs, 15k steps	99.9 %	1.86 ± 0.37	0.51 ± 0.01	21
TD3 (quadsim), 1024 envs, 15k steps	0.0 %	88.37 ± 1.94	0.62 ± 0.02	19

Table 19: New vehicles, control interfaces and tasks defined through the extension points (1000 episodes; position error in cm). Source: `vehicles_interfaces.json`.

Configuration	Success	Pos. err [cm]	Crash
hexa_x (6 rotors, generated frame), CTBR, PPO	99.8 %	1.21 ± 0.03	0.0 %
hexa_x, cascade baseline	96.5 %	4.40 ± 0.50	0.3 %

Table 20: Constrained waypoint racing: speed ≤ 2 m/s, budget 0.1 s of over-speed flight per episode (500 episodes). Source: `safe_rl.json`.

Policy	Success	Episode cost [s]	Time [s]	Crash
PPO (unconstrained)	99.4 %	5.309 ± 0.075	6.58 ± 0.10	0.6 %
cascade	92.8 %	6.476 ± 0.127	10.40 ± 0.16	2.2 %
PPO-Lagrangian (PID multiplier)	87.7 %	0.082 ± 0.017	11.86 ± 0.15	0.0 %

Table 21: Drone racing (2 laps, 7 gates, 5-inch racer); trained on random circuits. Source: `bench_race.json`.

Controller	Condition	success	lap_time	gates_passed	crash
cascade	random_tracks (id)	0.7 %	15.09 ± 0.33	5.72 ± 0.27	40.0 %
cascade	ellipse (ood)	0.0 %	15.00 ± 0.31	6.76 ± 0.27	21.1 %
cascade	figure8 (ood)	0.0 %	18.40 ± 0.17	5.91 ± 0.22	18.9 %
cascade	wind (ood)	0.7 %	15.20 ± 0.32	5.81 ± 0.27	38.6 %
ppo	random_tracks (id)	95.2 %	2.44 ± 0.01	13.49 ± 0.21	4.8 %
ppo	ellipse (ood)	99.6 %	2.49 ± 0.01	13.96 ± 0.06	0.4 %
ppo	figure8 (ood)	0.0 %	3.78	1.00 ± 0.09	100.0 %
ppo	wind (ood)	94.2 %	2.45 ± 0.02	13.42 ± 0.22	5.8 %

Table 22: Imitation learning. Hover: RL-to-BC distillation (last column: final position error [cm]). LangNav: success with training / held-out instruction phrasings and wrong-landmark rate (training phrasing). Source: `imitation.json`.

Task	Policy	Success (train phr.)	Success (held-out)	Wrong / pos. err
hover	PPO teacher	99.9 %	–	1.38 ± 0.26
hover	BC student	99.9 %	–	1.44 ± 0.38
hover	BC student, no last action	99.9 %	–	1.40 ± 0.21
langnav (map)	scripted expert (privileged)	–	100.0 %	–
langnav (map)	BC (with last action)	15.8 %	20.4 %	0.0 %
langnav (map)	BC, instruction removed	12.2 %	12.2 %	10.4 %
langnav (map)	BC, no last action	81.0 %	34.4 %	0.0 %
langnav (map)	+ unknown words ignored	81.0 %	65.3 %	0.0 %
langnav (map)	+ DAgger round 1	100.0 %	54.5 %	0.0 %
langnav (map)	+ DAgger round 2	100.0 %	59.7 %	0.0 %
langnav (map)	same data, frozen CLIP text encoder	100.0 %	99.8 %	0.0 %
langnav (RGB)	scripted expert (privileged)	100.0 %	–	0.0 %
langnav (RGB)	BC, FiLM-CNN + instruction	82.3 %	90.0 %	0.0 %
langnav (RGB)	+ DAgger round 1	100.0 %	100.0 %	0.0 %

Table 23: Extension examples executed in the simulator (`scripts/validation/examples_check.py`); success of the classical baseline where the check is an evaluation.

Example	Result	Baseline success
vehicle_yaml_hexa	pass	96 %
vehicle_yaml_inherit	pass	61 %
vehicle_plugin_quad250	pass	63 %
sensor_gnss	pass	–
task_landing	pass	–
task_landing_baseline	pass	0 %
reward_curriculum	pass	–
action_interface_accel	pass	–
algorithm_ars	pass	–
vla_adapter_color_servo	FAIL	–

yaw). Out of distribution, PPO is the most sensitive to latency beyond its training range (67 % vs 82–83 % for SAC and the BC student), so the benchmark exposes a property that the in-distribution number hides. TD3 with the same recipe did not learn (negative result).

- **Generality (Table 19).** A six-rotor vehicle generated from a frame description trains to 99.8 % with the unchanged task, and the extension examples (vehicle files, sensor terms, a new task, a new control interface, a new algorithm) run through the unmodified scripts.
- **Racing (Table 21).** PPO completes two laps on 95 % of unseen random circuits at 11.8 m/s mean speed (2.4 s per lap), transfers to a fixed reference circuit and to wind, but fails completely on the figure-8: the training circuits only turn counter-clockwise, the figure-8 alternates directions. The classical cascade with gate-aligned waypoints passes about 6 of 14 gates in the episode (15 s per lap, 40 % crashes); its first version (straight to the gate centre) crashed into the frames in 99 % of the episodes.
- **Constraints (Table 20).** The Lagrangian keeps the over-speed time below its budget (0.082 vs 0.1 s) while the unconstrained policy spends 5.3 s per course above the limit; the price is a slower course and a lower completion rate within the episode limit.
- **Imitation and language (Table 22).** An RL policy distils into a BC student without loss. For language-conditioned navigation, the last-action input makes BC copy its previous command and fail to stop (causal confusion [16]); removing it, ignoring unknown words and two DAgger rounds reach the expert's 100 %. A bag-of-words instruction encoder generalizes poorly to unseen phrasings (≈60 %), whereas a frozen pretrained CLIP text tower, with the same data and policy, reaches 99.8 %.

9 Limitations

The following limitations are explicit; none of them is hidden by the results above.

- **No real-world flight.** All results are in simulation, including PX4 SITL (Section 7); SITL shares the quadsim motor and drag parameters, so it does not validate the motor model. In SITL, navigation obstacles are virtual (no contact physics), the waypoint course is 25–35 % slower than in quadsim for a reason not yet isolated, depth/RGB policies have not been flown (no rendered gz camera matched to the training camera), and run counts are small (2–6 flights per configuration).
- **Model fidelity.** Motor and aerodynamic parameters come from the PX4 Gazebo X500 model and the Crazyflie literature, not from identification of a physical vehicle; body-drag coefficients are engineering estimates. Rotor aerodynamics use the standard quasi-static $T = k_f \omega^2$ model (no inflow/advance-ratio dependence, no blade-element or vortex-ring-state effects); the ground effect is applied per rotor but ceiling/wall effects are not modelled. Battery dynamics are represented only by a per-episode $\omega_{\max}$ scale.
- **PhysX discretization.** PhysX integrates at 200 Hz with a first-order scheme; relative to an RK4 reference this causes 3 mm to 9 mm position error over 1.5 s of open-loop flight (Table 2), negligible for closed-loop control but relevant for open-loop prediction studies (use the reference integrator).
- **Vision.** Depth policies are trained on analytic ray-cast depth of primitive obstacles with a parametric stereo-noise model; rendered depth agrees to 7 mm median, but real sensor artefacts (edge holes, speckle, IR interference, motion blur) are not modelled. RGB/RGB-D pipelines are implemented and execute end to end, but procedural scenes lack the texture/lighting diversity needed for transferable RGB policies.
- **Evaluation statistics.** The hover policies were trained with three seeds per training distribution (Section 7.5); the other tasks use a single seed (compute budget of an 8 GB consumer GPU), so their confidence intervals quantify evaluation noise, not seed-to-seed variability. Classical baselines use reasonable default gains, not systematically tuned gains, so the reported margins are upper bounds of the RL advantage.
- **Framework scope.** The racing, multi-agent, safe-RL, landing and language tasks are trained with one seed; presets other than the X500 and the Crazyflie (5-inch racer, hexarotor) are

engineering approximations. The language encoder is a small bag-of-words model trained from eight instruction templates; no pretrained VLM/VLA was run (the adapter interface and the LeRobot export are validated, external models are not). TD3 with the SAC recipe did not learn hover. Autonomous exploration and explicit adaptive-control tasks are not implemented (observation histories are available as terms). The Lagrangian is the reward-penalty form without a cost critic.

- **Platform issues.** Isaac Lab v2.3.2 fails to initialize rigid bodies when a viewport is rendered (GUI, video); quadsim applies a version-gated runtime patch (Section 6.3.1). Interactive GUI and 4096-environment training do not fit simultaneously in 8 GB VRAM. The first launch with rendering enabled crashed once in the shader compiler (empty cache); later launches succeeded.

10 Roadmap toward real-world deployment

Prioritized by expected impact on a successful first real flight:

1. **PX4 SITL in the loop** – done (Section 7). Follow-ups: train velocity-interface policies with an identified or randomized autopilot velocity loop; flight-test the thrust-curve map on hardware; retrain tracking and waypoint policies on the sim-to-real distribution; a rendered gz camera matched to the training camera for depth policies; HITL with the companion computer to measure the real latency chain.
2. **System identification of the target vehicle**: mass and inertia (bifilar pendulum), k_f, k_m and motor time constants (thrust stand), rotor-drag coefficient (flight logs); update the preset and center the DR ranges on the identified values; retrain with several seeds.
3. **Latency identification** of the full pipeline (estimator, companion computer, uXRCE-DDS, PX4) and matching the delay distribution used in training; add estimator-in-the-loop training using recorded EKF2 error statistics.
4. **Hardware hover/tracking experiments** (tethered, then free flight in a motion-capture arena) with the safety layer (geofence, odometry watchdog, offboard-loss failsafe), logging with the same HDF5 schema for sim/real comparison.
5. **Depth-based navigation transfer**: record real depth frames, extend the depth noise model or add image-space randomization, distill the state/range teacher into the depth student (DAgger) to reduce sample cost, then test in a real obstacle course.
6. **Framework**: multi-seed results for the new tasks and algorithms; a cost critic for constrained RL; pretrained text/vision encoders and fine-tuning of an open VLA on the exported LeRobot data; texture and lighting randomization for RGB policies; exploration and adaptive-control benchmark tasks; recurrent policies for implicit system identification; migration to Isaac Lab 3.x once stable.

11 Reproducibility

- Software: Isaac Sim 5.1.0 (pip), Isaac Lab v2.3.2 (`third_party/IsaacLab`), Python 3.11, torch 2.7.0+cu128, warp-lang 1.11.1, rsl-rl-lib 3.1.2, skrl 1.4.3, onnxruntime 1.20.1 (all pins in `env/constraints.txt`); native installation with `scripts/setup_env.sh`; container `docker/Dockerfile` on the official image `nvcr.io/nvidia/isaac-lab:2.3.2`.
- Every training run stores the exact environment and agent configuration (YAML), command line, git commit and diff summary next to its checkpoints; evaluation JSONs store seeds and overrides.
- Framework checks: `scripts/validation/regression_evals.sh` + `compare_evals.py` (bit-exact reference metrics after core changes), `scripts/validation/examples_check.py` (every extension example in the simulator), `scripts/benchmark/run.py` (suites), `scripts/tools/collect_framework_results.py` (curated results in `docs/results/framework`); LeRobot exports are verified by `scripts/il/export_lerobot.py` in the optional `third_party/lerobot-venv`.
- Validation: `pytest tests/core` (unit tests), `scripts/validation/m{1,2,3,6}_*.py` (milestone acceptance tests), `scripts/rl/evaluate.py` (policy evaluation); tables in this report

are generated from `docs/results/*.json` by `scripts/tools/report_tables.py`.

- PX4 SITL: `ros2/px4_sitl/setup_px4_sitl.sh` (PX4 v1.16.2, ROS 2 Jazzy, Gazebo Harmonic, px4_msgs from the same PX4 tree), `scripts/sitl/run_sitl.sh` (one experiment from fresh PX4 parameters to landing), the metric scripts `compare.py`, `compare_tasks.py`, `estimator_stats.py` and `hover_v1_summary.py` in `scripts/sitl/`; the raw recordings (ROS-side `.npz`, bridge step logs, PX4 `.ulg`) are kept in `logs/sitl/`.

References

[1] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, D. Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature* 620, 2023.

[2] J. Eschmann, D. Albani, G. Loianno. Learning to fly in seconds. *IEEE RA-L*, 2024.

[3] E. Kaufmann, L. Bauersfeld, D. Scaramuzza. A benchmark comparison of learned control policies for agile quadrotor flight. *ICRA*, 2022.

[4] M. Kulkarni, W. Rehberg, K. Alexis. Aerial Gym Simulator: A framework for highly parallelized simulation of aerial robots. *IEEE RA-L*, 2025.

[5] B. Xu et al. OmniDrones: An efficient and flexible platform for reinforcement learning in drone control. *IEEE RA-L*, 2024.

[6] T. Lee, M. Leok, N. H. McClamroch. Geometric tracking control of a quadrotor UAV on SE(3). *IEEE CDC*, 2010.

[7] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, P. Abbeel. Asymmetric actor critic for image-based robot learning. *RSS*, 2018.

[8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal policy optimization algorithms. arXiv:1707.06347, 2017.

[9] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine. Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor. *ICML*, 2018.

[10] S. Fujimoto, H. van Hoof, D. Meger. Addressing function approximation error in actor-critic methods. *ICML*, 2018.

[11] Y. Seo, C. Sferrazza, H. Geng, M. Nauman, Z.-H. Yin, P. Abbeel. FastTD3: simple, fast, and capable reinforcement learning for humanoid control. arXiv, 2025.

[12] A. Stooke, J. Achiam, P. Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. *ICML*, 2020.

[13] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, Y. Wu. The surprising effectiveness of PPO in cooperative multi-agent games. *NeurIPS Datasets and Benchmarks*, 2022.

[14] Y. Song, M. Steinweg, E. Kaufmann, D. Scaramuzza. Autonomous drone racing with deep reinforcement learning. *IROS*, 2021.

[15] S. Ross, G. Gordon, D. Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. *AISTATS*, 2011.

[16] P. de Haan, D. Jayaraman, S. Levine. Causal confusion in imitation learning. *NeurIPS*, 2019.

[17] E. Perez, F. Strub, H. de Vries, V. Dumoulin, A. Courville. FiLM: visual reasoning with a general conditioning layer. *AAAI*, 2018.

[18] H. Mania, A. Guy, B. Recht. Simple random search of static linear policies is competitive for reinforcement learning. *NeurIPS*, 2018.

[19] R. Cadene et al. LeRobot: state-of-the-art machine learning for real-world robotics in PyTorch. Software, `https://github.com/huggingface/lerobot`, 2024.

[20] J. Förster. System identification of the Crazyflie 2.0 nano quadrocopter. BSc thesis, ETH Zurich, 2015.